DATA SCIENCE PORTFOLIO PROJECT REPORT

FitPulse 360

An Integrated Gym, Fitness, and Activity Data Analytics Project



Exploratory Data Analysis | Data Cleaning | Feature Engineering | Data Visualization

PREPARED BY

Sarim Mohammed Maqsood Ahmed

TECHNOLOGIES USED

Python | Pandas | NumPy | Matplotlib | Seaborn | Google Colab

Dataset: Gym Members Exercise Dataset (Kaggle) | 973 records | 15 variables

Environment: Python and Google Colab | Report date: 17 October 2026

Table of Contents

1. Abstract and Executive Summary . . . 3
2. Problem Statement . . . 3
3. Project Objectives . . . 4
4. Dataset Description . . . 4
5. Tools and Technologies . . . 5
6. Methodology . . . 6
7. Data Cleaning . . . 6
8. Exploratory Data Analysis . . . 8
9. Charts and Visualizations . . . 10
10. Feature Engineering . . . 15
11. Outlier Detection . . . 18
12. Actual Findings and Results . . . 20
13. Source Code Appendix . . . 21
14. Code Outputs . . . 29
15. Dashboard . . . 33
16. Project Limitations . . . 34
17. Future Enhancements . . . 34
18. Conclusion . . . 35
19. References . . . 35

About this report

Every number, table and chart in this report was calculated by running the project's Python script (`FitPulse_360_Analysis.py`) on the uploaded file `gym_members_exercise_tracking.csv` (973 rows, 15 columns). Nothing was typed in by hand. The script was executed in Python 3.12.3 with pandas 3.0.2, NumPy 2.4.4, Matplotlib 3.10.8 and seaborn 0.13.2; it was not run inside Google Colab itself, so the Colab steps in Section 13 are given as instructions rather than as an executed result.

1. Abstract and Executive Summary

FitPulse 360 is a beginner-level Data Science portfolio project that analyses the Gym Members Exercise Dataset from Kaggle. The dataset describes **973 gym members** using **15 variables**: demographics, body measurements, heart-rate readings, session duration, calories burned, workout type and lifestyle habits. The project follows a complete and reproducible workflow in Python: data loading and inspection, cleaning and validation, exploratory statistics, visualisation, feature engineering, outlier detection, group comparison and export of the results.

The purpose of the project is to turn raw fitness records into clear, evidence-based answers: Is the data trustworthy? What does the typical member look like? How many calories are burned per session, and what drives that number? The work demonstrates the core skills of a junior data analyst and forms the foundation for a wider FitPulse 360 platform that could later include dance, cardio and real activity-tracker data.

Key results at a glance

Measure	Result
Dataset size	973 rows x 15 columns; 19 columns after feature engineering
Data quality	0 missing values, 0 duplicate rows; 23 records (2.4%) have Avg_BPM above Max_BPM and are flagged
Most common workout	Strength (258 members, 26.5%)
Average calories burned	905.42 kcal per session
Average session duration	1.26 hours
Average BMI / heart rate	BMI 24.91; average 143.77 BPM during sessions
Duration vs calories	Pearson r = 0.908 (strong positive); about 722 kcal per extra hour
Outliers (IQR)	54 of 973 rows have at least one flagged value; all were retained after inspection

Significance. The analysis shows that session length is by far the strongest single driver of calories burned in this dataset, that the four workout types differ very little in average calories (a spread of only 4.6% of the overall mean, not statistically significant), and that men burn more calories than women on average despite nearly identical session lengths. It also documents honestly where the data has weaknesses, which is an important habit for any data scientist.

2. Problem Statement

Gyms, trainers and fitness apps collect large amounts of information such as heart rate, session length and calories burned, but raw tables of numbers do not by themselves tell anyone what is happening. Without systematic analysis, several problems appear: data-quality issues go unnoticed, averages hide differences between groups, and strong-looking patterns are mistaken for cause and effect. A structured analysis turns the records into trustworthy summaries that members, coaches and future machine-learning models can use.

Questions investigated

#	Research question
RQ1	Is the dataset clean and reliable (missing values, duplicates, logical consistency)?
RQ2	What does the typical member look like (workout mix, gender, experience, BMI, heart rate)?
RQ3	How many calories are burned per session and how does this vary by workout type, gender and experience?
RQ4	How strongly are session duration and calories burned related?
RQ5	Do engineered features (BMI category, calories per hour, duration category, water per kg) add useful insight?
RQ6	Are there outliers, and should they be removed?

The answers are summarised in Section 12 (Actual Findings and Results).

3. Project Objectives

Each objective is stated with a measurable success criterion and the result achieved when the project was executed.

#	Objective	Measurable criterion	Result
1	**Load and inspect the dataset**	Report shape, data types and unique categories	973 x 15; 7 integer, 6 floating-point and 2 text
2	**Assess data quality**	Check all 15 columns for missing values and all rows for duplicates	0 missing, 0 duplicates; 5 validity checks run
3	**Compute descriptive statistics**	Summary statistics for all numeric columns; counts for all categories	13 numeric columns, 3 category tables
4	**Visualise the data**	At least 10 labelled charts plus one dashboard	13 charts + 1 dashboard (14 figures)
5	**Engineer new features**	Create exactly four features	BMI_Category, Calories_Per_Hour, Workout_Duration_Category, Water_Intake_Per_Kg
6	**Detect outliers**	Apply the IQR rule to every continuous variable	13 variables tested; 54 rows flagged
7	**Quantify relationships**	Pearson correlation matrix and significance tests	12 x 12 matrix; ANOVA, Welch t-test, Pearson test
8	**Export reproducible results**	Save cleaned and feature-engineered CSV files	3 CSV files exported and re-loaded successfully

4. Dataset Description

Source: Gym Members Exercise Dataset on Kaggle (uploaded by *valakhorasani*): https://www.kaggle.com/datasets/valakhorasani/gym-members-exercise-dataset. **File analysed:** `gym_members_exercise_tracking.csv`.

Property	Value
Rows (records)	973
Columns (variables)	15 in the original file; 19 after feature engineering
Data types	7 integer, 6 floating-point and 2 text columns (Gender and Workout_Type are text)
Missing values	0
Duplicate rows	0
Categorical values	Gender: Female, Male. Workout_Type: Cardio, HIIT, Strength, Yoga. Experience_Level: 1, 2, 3

Column dictionary (observed ranges calculated from the data)

#	Column	Type	Description	Observed range
1	**Age**	int	Age of the member in years	18 to 59
2	**Gender**	text	Gender of the member	Female, Male
3	**Weight (kg)**	float	Body weight in kilograms	40 to 129.9
4	**Height (m)**	float	Height in metres	1.5 to 2
5	**Max_BPM**	int	Maximum heart rate during the session (beats per minute)	160 to 199
6	**Avg_BPM**	int	Average heart rate during the session	120 to 169
7	**Resting_BPM**	int	Heart rate at rest before the session	50 to 74
8	**Session_Duration (hours)**	float	Length of the workout session in hours	0.5 to 2
9	**Calories_Burned**	int	Total calories burned in the session (kcal)	303 to 1783
10	**Workout_Type**	text	Type of workout performed	Cardio, HIIT, Strength, Yoga
11	**Fat_Percentage**	float	Body-fat percentage	10 to 35
12	**Water_Intake (liters)**	float	Daily water intake in litres	1.5 to 3.7
13	**Workout_Frequency (days/week)**	int	Number of workout days per week	2 to 5
14	**Experience_Level**	int	Ordinal experience scale, 1 (beginner) to 3 (expert)	1 to 3
15	**BMI**	float	Body Mass Index = weight / height squared	12.32 to 49.84

Relevant limitations of the dataset

- The file contains one row per member with no dates or member IDs, so progress over time cannot be studied.
- Several variables sit inside tight bounds (Age 18 to 59, Resting_BPM 50 to 74, Fat_Percentage 10 to 35). The collection method is not documented in the file, so the data may be curated or simulated rather than a random sample of gym members.
- How calories were measured (device, formula or estimate) is not stated.
- 23 records report an average heart rate above the maximum heart rate, which is logically inconsistent (Section 7).

Section 16 discusses these limitations in more detail.

5. Tools and Technologies

Tool	Version used	How it is used in FitPulse 360
Python	3.12.3	Programming language for the whole analysis; written as clearly labelled steps.
Pandas	3.0.2	Loading the CSV, inspecting data, cleaning, groupby summaries, pivot tables, pd.cut for feature engineering, exporting CSV files.
NumPy	2.4.4	Numeric operations: infinity bin edges, linear fit (polyfit), linspace for the trend line, numeric type selection.
Matplotlib	3.10.8	Figure layout, axes, annotations, dashboard grid and saving high-resolution PNG images.
Seaborn	0.13.2	Statistical charts: count plots, histograms with KDE, box plots, scatter plots and the correlation heatmap.
Google Colab	Cloud notebook	Target environment: the script is split into cells with # %% markers so it can be pasted into a Colab notebook.
SciPy (optional)	1.17.1	Used only for three significance tests in Step 4 (ANOVA, Welch t-test, Pearson test).

Tool	Version used	How it is used in FitPulse 360
ReportLab	4.4.10	Builds this PDF report from the executed results.

6. Methodology

The project follows a linear, reproducible pipeline. Each stage is one step in the script, and each step's printed output and charts are included in this report.

Stage	Operation	Method / function	Script step
1	**Data loading**	pd.read_csv, shape and column names	Step 1
2	**Data inspection**	head, info, dtypes, unique values	Step 2
3	**Missing-value treatment**	isnull().sum(); median / mode rule if gaps exist	Step 3
4	**Duplicate removal**	duplicated(), drop_duplicates()	Step 3
5	**Validation checks**	Logical rules and BMI recomputation	Step 3
6	**Exploratory analysis and statistical summaries**	describe, value_counts, groupby, correlations, significance tests	Step 4
7	**Visualisation**	Eight seaborn / Matplotlib charts	Step 5
8	**Feature engineering**	pd.cut and arithmetic on columns (4 features)	Step 6
9	**Outlier detection**	IQR rule: Q1 - 1.5 x IQR and Q3 + 1.5 x IQR	Step 7
10	**Group comparisons**	groupby, pivot_table, comparison charts	Step 8
11	**Dashboard**	GridSpec layout with KPI strip and six panels	Step 9
12	**Exporting results**	to_csv and re-load consistency check	Step 10
13	**Findings summary**	All headline numbers printed from the data	Step 11

Design principle: the cleaning step works on a copy (`df_clean`) and the feature-engineering step works on a second copy (`df_fe`), so the raw data in `df` is never modified.

7. Data Cleaning

The cleaning step checks every column for missing values, removes duplicates, standardises text, tests logical rules and verifies that the stored BMI agrees with weight and height. The complete code (Step 3) and its real output are shown below.

PYTHON CODE | STEP 3

```
df_clean = df.copy()

# 3.1 Missing values (imputation runs only if a column actually has gaps)
missing = df_clean.isnull().sum()
print("3.1 Missing values per column")
print(missing.to_string())
print("Total missing values:", int(missing.sum()))
for col in df_clean.columns:
    if df_clean[col].isnull().any():
        if pd.api.types.is_numeric_dtype(df_clean[col]):
            df_clean[col] = df_clean[col].fillna(df_clean[col].median())
        else:
            df_clean[col] = df_clean[col].fillna(df_clean[col].mode()[0])
print("Missing values after treatment:", int(df_clean.isnull().sum().sum()))

# 3.2 Duplicate records
n_dup = int(df_clean.duplicated().sum())
print("\n3.2 Duplicate rows found:", n_dup)
df_clean = df_clean.drop_duplicates().reset_index(drop=True)
print("Rows after duplicate removal:", len(df_clean))

# 3.3 Text standardisation (remove accidental leading/trailing spaces)
```

```
text_cols = [c for c in df_clean.columns if not pd.api.types.is_numeric_dtype(df_clean[c])]
for col in text_cols:
    df_clean[col] = df_clean[col].astype(str).str.strip()
print("\n3.3 Text columns standardised:", text_cols)

# 3.4 Logical validity checks (0 means no problem found)
print("\n3.4 Logical validity checks")
checks = {
    "Non-positive numeric values": int((df_clean.select_dtypes("number") <= 0).sum().sum()),
    "Avg_BPM greater than Max_BPM": int((df_clean["Avg_BPM"] > df_clean["Max_BPM"]).sum()),
    "Resting_BPM greater than Avg_BPM": int((df_clean["Resting_BPM"] > df_clean["Avg_BPM"]).sum()),
    "Fat_Percentage outside 0-100": int((~df_clean["Fat_Percentage"].between(0, 100)).sum()),
}
for name, value in checks.items():
    print(f"{name:<36}: {value}")

# Inspect (but do not delete) records where average BPM exceeds maximum BPM
bpm_conflict = df_clean[df_clean["Avg_BPM"] > df_clean["Max_BPM"]]
print("\nRecords where Avg_BPM > Max_BPM (first 5 of", len(bpm_conflict), "):")
print(bpm_conflict[["Age", "Gender", "Max_BPM", "Avg_BPM", "Resting_BPM", "Workout_Type"]]
      .head(5).to_string())
print("Decision: flagged and retained - the source of the inconsistency cannot be verified.")

# 3.5 BMI consistency: recompute BMI = weight / height^2 and compare
bmi_check = (df_clean["Weight (kg)"] / df_clean["Height (m)"] ** 2).round(2)
bmi_gap = (bmi_check - df_clean["BMI"]).abs()
print(f"\n3.5 BMI recomputed from weight and height: max abs difference = {bmi_gap.max():.2f}")
print("Rows with a difference above 0.05:", int((bmi_gap > 0.05).sum()))

# 3.6 Export the cleaned dataset
df_clean.to_csv("fitpulse_cleaned_dataset.csv", index=False)
print("\nSaved: fitpulse_cleaned_dataset.csv", df_clean.shape)
```

OUTPUT | TABULAR

```
3.1 Missing values per column
Age                              0
Gender                           0
Weight (kg)                      0
Height (m)                       0
Max_BPM                          0
Avg_BPM                          0
Resting_BPM                      0
Session_Duration (hours)         0
Calories_Burned                  0
Workout_Type                     0
Fat_Percentage                   0
Water_Intake (liters)            0
Workout_Frequency (days/week)    0
Experience_Level                 0
BMI                              0
Total missing values: 0
Missing values after treatment: 0
```

OUTPUT | TEXT

```
3.2 Duplicate rows found: 0
Rows after duplicate removal: 973
```

OUTPUT | TEXT

```
3.3 Text columns standardised: ['Gender', 'Workout_Type']
```

OUTPUT | TEXT

```
3.4 Logical validity checks
Non-positive numeric values         : 0
Avg_BPM greater than Max_BPM        : 23
Resting_BPM greater than Avg_BPM    : 0
Fat_Percentage outside 0-100        : 0
```

OUTPUT | TABULAR

```
Records where Avg_BPM > Max_BPM (first 5 of 23 ):
     Age  Gender  Max_BPM  Avg_BPM  Resting_BPM Workout_Type
16    19  Female      166      167           58         HIIT
46    24    Male      160      167           53         Yoga
56    59  Female      161      162           61         Yoga
95    59  Female      164      169           64       Cardio
106   41    Male      162      168           51         Yoga
```

OUTPUT | TEXT

```
Decision: flagged and retained - the source of the inconsistency cannot be verified.
```

OUTPUT | TEXT

```
3.5 BMI recomputed from weight and height: max abs difference = 0.01
Rows with a difference above 0.05: 0

Saved: fitpulse_cleaned_dataset.csv (973, 15)
```

What each operation does

- **3.1 Missing values:** `isnull().sum()` counts gaps per column. The rule (median for numbers, mode for text) is written so the script is reusable, but it did not trigger because no gaps exist.
- **3.2 Duplicates:** `duplicated()` finds identical rows; none were found, so the row count stays at 973.
- **3.3 Text standardisation:** strips stray spaces from Gender and Workout_Type so that categories are never split by invisible characters.
- **3.4 Validity checks:** no non-positive values, no resting heart rate above the average, no impossible fat percentages. **However, 23 records have Avg_BPM greater than Max_BPM**, which cannot be true physiologically.
- **3.5 BMI check:** BMI recomputed as weight / height squared matches the stored column (maximum difference 0.01, caused by rounding).

> **Data-quality decision**
>
> The 23 heart-rate conflicts (2.4% of rows) were **flagged and retained**, not deleted. Their calorie, duration and body-measurement values look normal, and there is no way to tell whether Avg_BPM or Max_BPM is the wrong value. Heart-rate averages in this report should therefore be read with that caveat.

8. Exploratory Data Analysis

Step 4 calculates descriptive statistics, category counts, workout-type averages, correlations with Calories_Burned and three significance tests. Full code and output follow.

PYTHON CODE | STEP 4

```
num_cols = df_clean.select_dtypes(include="number").columns.tolist()

print("4.1 Descriptive statistics")
print(df_clean[num_cols].describe().T.round(2).to_string())

print("\n4.2 Category counts")
for col in ["Workout_Type", "Gender", "Experience_Level"]:
    counts = df_clean[col].value_counts()
    pct = (counts / len(df_clean) * 100).round(1)
    print(f"\n{col}")
    print(pd.DataFrame({"Count": counts, "Percent": pct}).to_string())

print("\n4.3 Average values by workout type")
by_workout = df_clean.groupby("Workout_Type")[
    ["Calories_Burned", "Session_Duration (hours)", "Avg_BPM", "BMI"]].mean().round(2)
print(by_workout.sort_values("Calories_Burned", ascending=False).to_string())

print("\n4.4 Correlation of every numeric column with Calories_Burned")
cal_corr = df_clean[num_cols].corr()["Calories_Burned"].drop("Calories_Burned")
print(cal_corr.sort_values(ascending=False).round(3).to_string())

def fmt_p(p):
```

```
    return "< 0.001" if p < 0.001 else f"= {p:.4f}"

print("\n4.5 Significance checks (alpha = 0.05)")
groups = [g["Calories_Burned"].values for _, g in df_clean.groupby("Workout_Type")]
f_stat, p_anova = stats.f_oneway(*groups)
print(f"One-way ANOVA, calories by workout type : F = {f_stat:.3f}, p {fmt_p(p_anova)}")
male = df_clean.loc[df_clean["Gender"] == "Male", "Calories_Burned"]
female = df_clean.loc[df_clean["Gender"] == "Female", "Calories_Burned"]
t_stat, p_gender = stats.ttest_ind(male, female, equal_var=False)
print(f"Welch t-test, calories male vs female   : t = {t_stat:.3f}, p {fmt_p(p_gender)}")
r_val, p_corr = stats.pearsonr(df_clean["Session_Duration (hours)"], df_clean["Calories_Burned"])
print(f"Pearson test, duration vs calories      : r = {r_val:.3f}, p {fmt_p(p_corr)}")
```

OUTPUT | TABULAR

```
4.1 Descriptive statistics
                              count    mean     std     min     25%     50%      75%      max
Age                           973.0   38.68   12.18   18.00   28.00   40.00    49.00    59.00
Weight (kg)                   973.0   73.85   21.21   40.00   58.10   70.00    86.00   129.90
Height (m)                    973.0    1.72    0.13    1.50    1.62    1.71     1.80     2.00
Max_BPM                       973.0  179.88   11.53  160.00  170.00  180.00   190.00   199.00
Avg_BPM                       973.0  143.77   14.35  120.00  131.00  143.00   156.00   169.00
Resting_BPM                   973.0   62.22    7.33   50.00   56.00   62.00    68.00    74.00
Session_Duration (hours)      973.0    1.26    0.34    0.50    1.04    1.26     1.46     2.00
Calories_Burned               973.0  905.42  272.64  303.00  720.00  893.00  1076.00  1783.00
Fat_Percentage                973.0   24.98    6.26   10.00   21.30   26.20    29.30    35.00
Water_Intake (liters)         973.0    2.63    0.60    1.50    2.20    2.60     3.10     3.70
Workout_Frequency (days/week) 973.0    3.32    0.91    2.00    3.00    3.00     4.00     5.00
Experience_Level              973.0    1.81    0.74    1.00    1.00    2.00     2.00     3.00
BMI                           973.0   24.91    6.66   12.32   20.11   24.16    28.56    49.84
```

OUTPUT | TABULAR

```
4.2 Category counts

Workout_Type
              Count  Percent
Workout_Type
Strength        258     26.5
Cardio          255     26.2
Yoga            239     24.6
HIIT            221     22.7

Gender
        Count  Percent
Gender
Male      511     52.5
Female    462     47.5

Experience_Level
                  Count  Percent
Experience_Level
2                   406     41.7
1                   376     38.6
3                   191     19.6
```

OUTPUT | TABULAR

```
4.3 Average values by workout type
              Calories_Burned  Session_Duration (hours)  Avg_BPM    BMI
Workout_Type
HIIT                   925.81                      1.29   143.52  25.20
Strength               910.70                      1.26   144.31  24.54
Yoga                   903.19                      1.26   143.27  24.55
Cardio                 884.51                      1.22   143.89  25.38
```

OUTPUT | TABULAR

```
4.4 Correlation of every numeric column with Calories_Burned
Session_Duration (hours)         0.908
Experience_Level                 0.694
Workout_Frequency (days/week)    0.576
Water_Intake (liters)            0.357
Avg_BPM                          0.340
Weight (kg)                      0.095
Height (m)                       0.086
BMI                              0.060
Resting_BPM                      0.017
Max_BPM                          0.002
Age                             -0.155
```

```
Fat_Percentage                -0.598
```

OUTPUT | TEXT

```
4.5 Significance checks (alpha = 0.05)
One-way ANOVA, calories by workout type : F = 0.949, p = 0.4162
Welch t-test, calories male vs female   : t = 4.781, p < 0.001
Pearson test, duration vs calories      : r = 0.908, p < 0.001
```

Interpretation

- **Calories:** mean 905.42 kcal, median 893, standard deviation 272.64; values range from 303 to 1,783. Mean slightly above median indicates mild right skew (skewness 0.28).
- **Workout mix is balanced:** Strength 26.5%, Cardio 26.2%, Yoga 24.6%, HIIT 22.7%. Members are 52.5% male and 47.5% female.
- **Strongest correlates of calories:** session duration (r = 0.908), experience level (0.694), workout frequency (0.576); body-fat percentage is negatively correlated (-0.598).
- **Significance checks:** workout type does **not** significantly change average calories (ANOVA F = 0.949, p = 0.42); men and women differ significantly (Welch t = 4.78, p < 0.001); the duration-calories correlation is highly significant (p < 0.001).

9. Charts and Visualizations

The eight charts below were generated by Step 5 from the cleaned data. Each has a title, labelled axes, and a legend where more than one series is shown. The code for every chart is in the Source Code Appendix (Section 13, Step 5).

OUTPUT | VISUAL

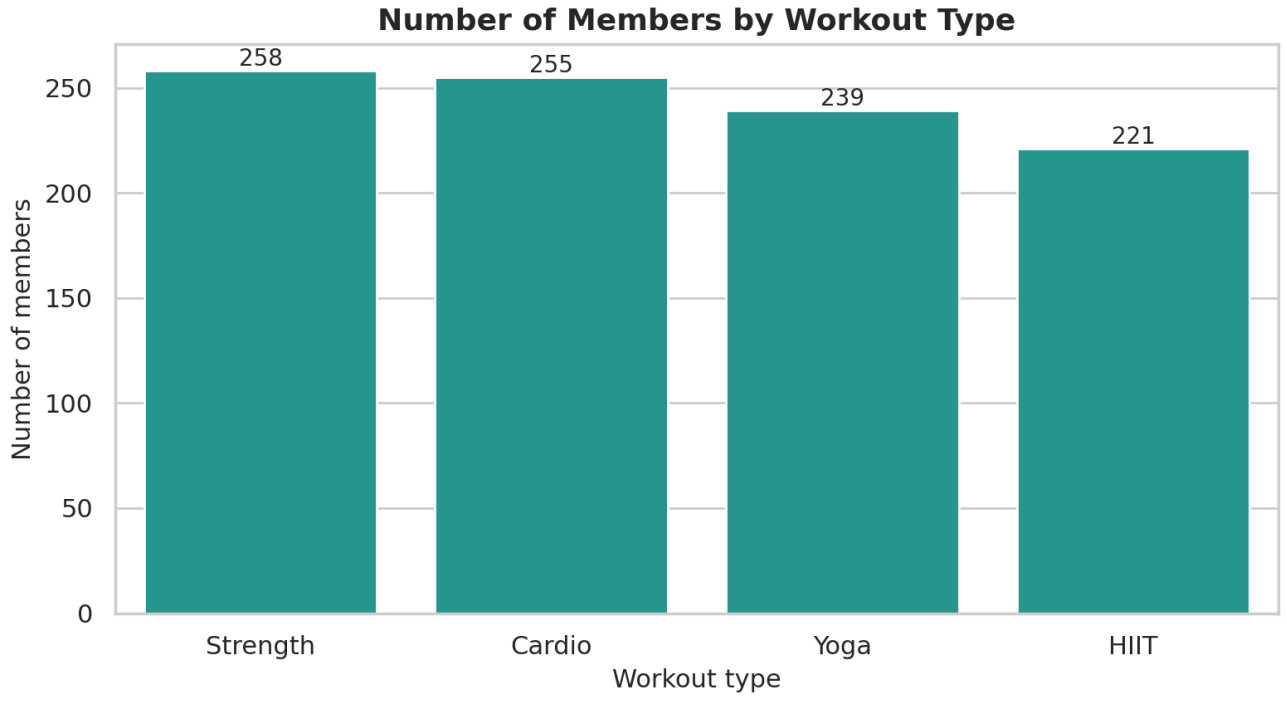


Figure 1. Number of members by workout type

What it shows: Strength (258), Cardio (255), Yoga (239) and HIIT (221) are all within 37 members of each other. The sample is balanced, so group averages are not distorted by one dominant workout type.

OUTPUT | VISUAL

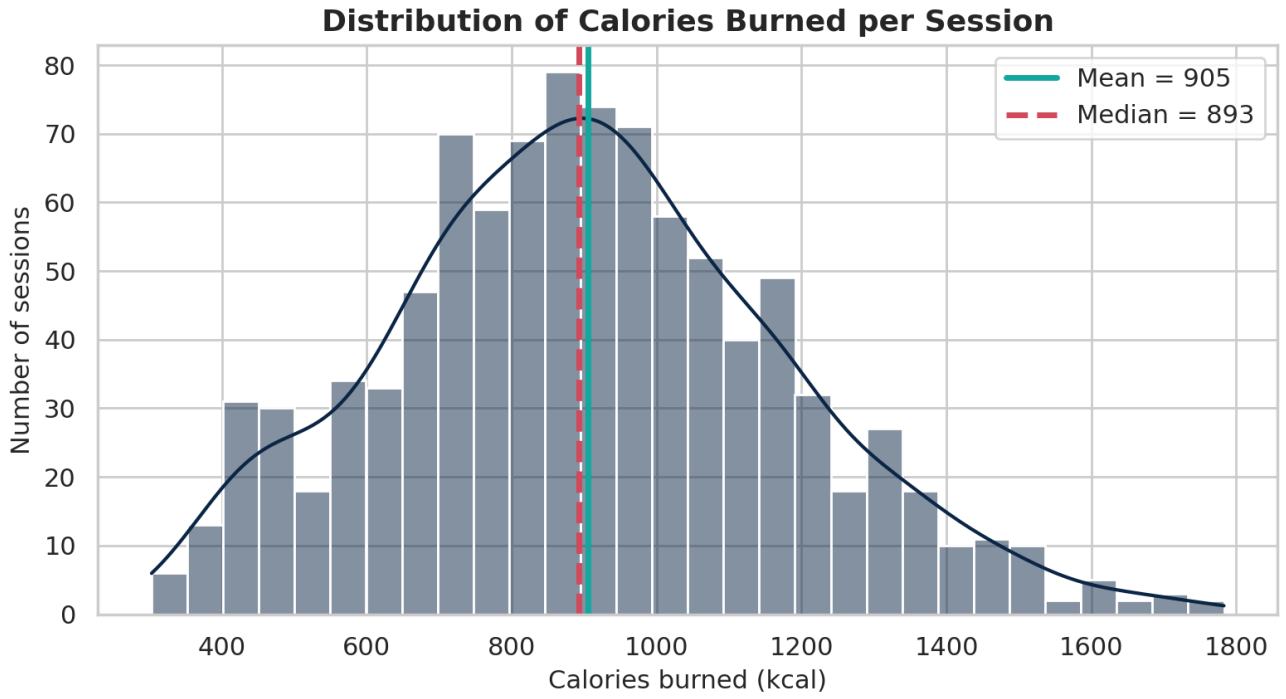


Figure 2. Distribution of calories burned per session

What it shows: A single-peaked, slightly right-skewed distribution. The mean (905 kcal) sits just above the median (893 kcal); a few long sessions pull the tail towards 1,783 kcal.

OUTPUT | VISUAL

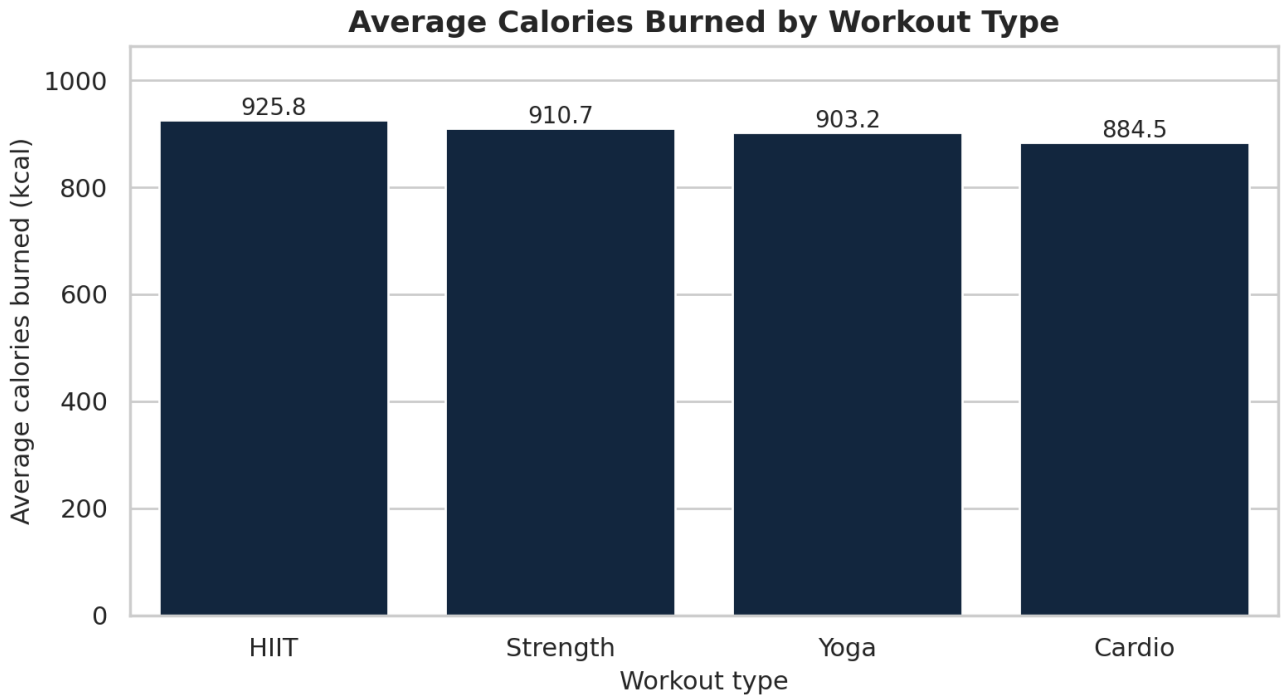


Figure 3. Average calories burned by workout type

What it shows: HIIT is highest (925.8 kcal) and Cardio lowest (884.5 kcal), a gap of only 41.3 kcal (4.6% of the overall mean). The ANOVA in Step 4 shows this gap is not statistically significant (p = 0.42).

OUTPUT | VISUAL

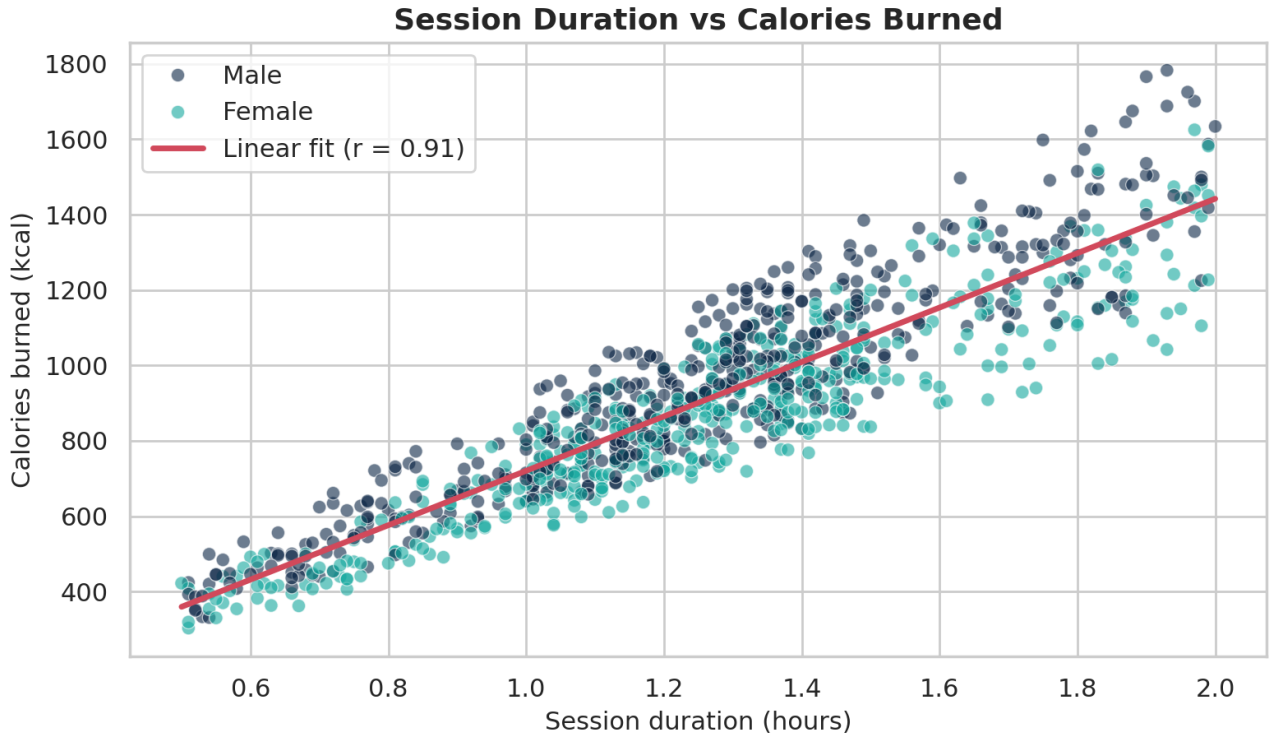


Figure 4. Session duration versus calories burned

What it shows: A clear upward linear trend (r = 0.908, R-squared = 0.825). Each extra hour adds about 722 kcal on average. At the same duration, male members (dark points) tend to sit above female members (teal points).

OUTPUT | VISUAL

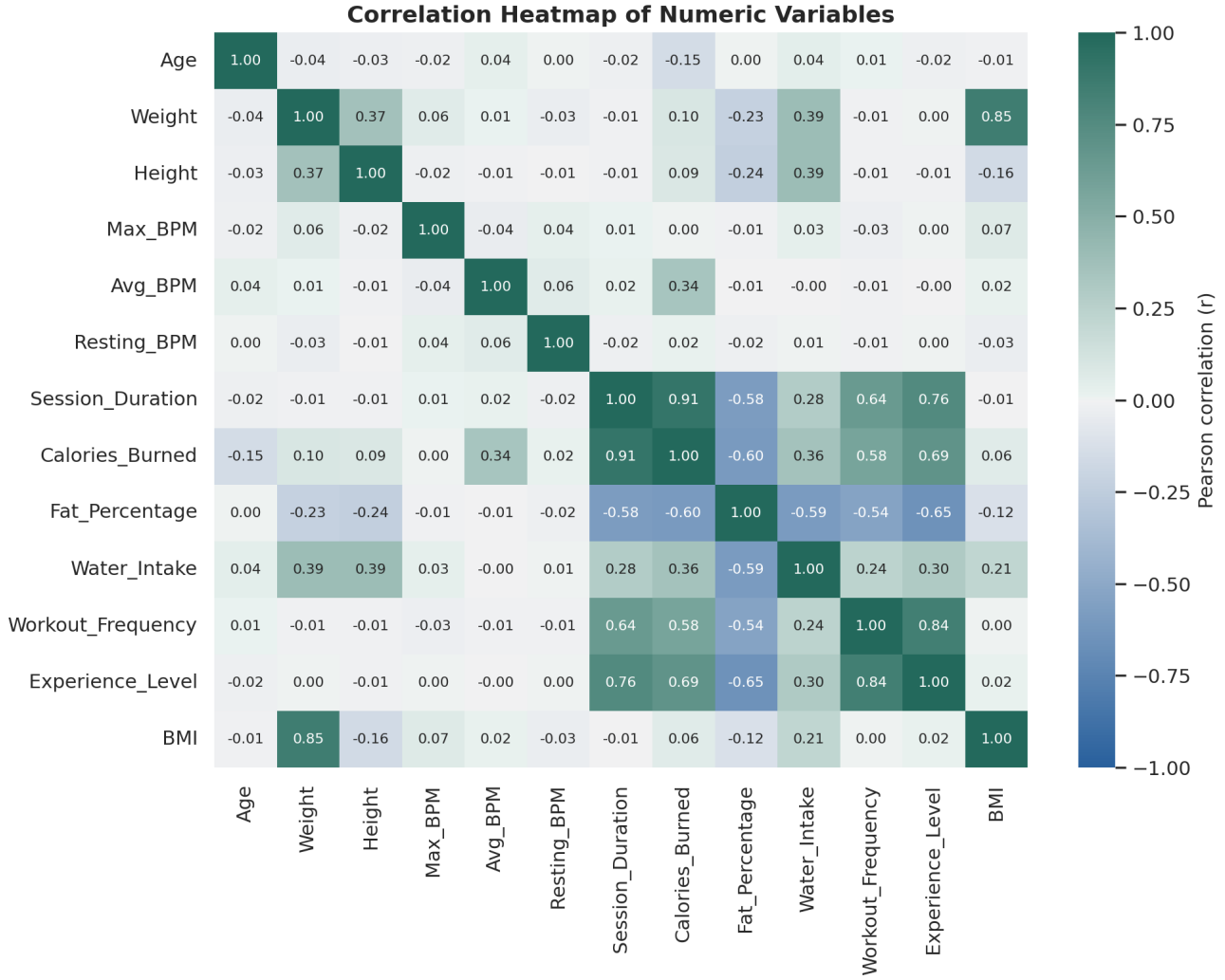


Figure 5. Correlation heatmap of numeric variables

What it shows: Strongest relationships: Session_Duration and Calories_Burned (0.91), Weight and BMI (0.85), Experience_Level and Workout_Frequency (0.84), Experience_Level and Session_Duration (0.76). Fat_Percentage is negatively related to Experience_Level (-0.65) and Calories_Burned (-0.60). Max_BPM and Resting_BPM are almost unrelated to everything else.

OUTPUT | VISUAL

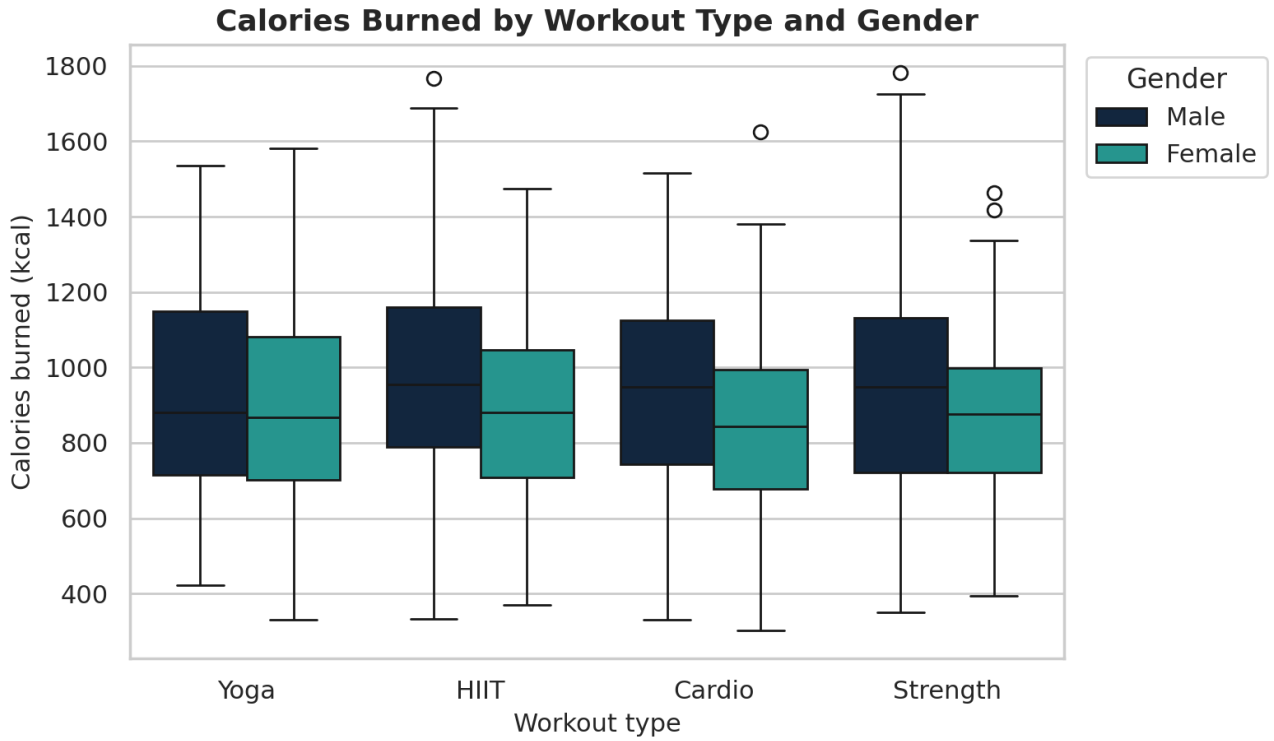


Figure 6. Calories burned by workout type and gender

What it shows: Male members burn more calories on average in every workout type. The gap is largest for HIIT (108 kcal) and smallest for Yoga (31 kcal). The boxes overlap strongly, so many women burn more than many men.

OUTPUT | VISUAL

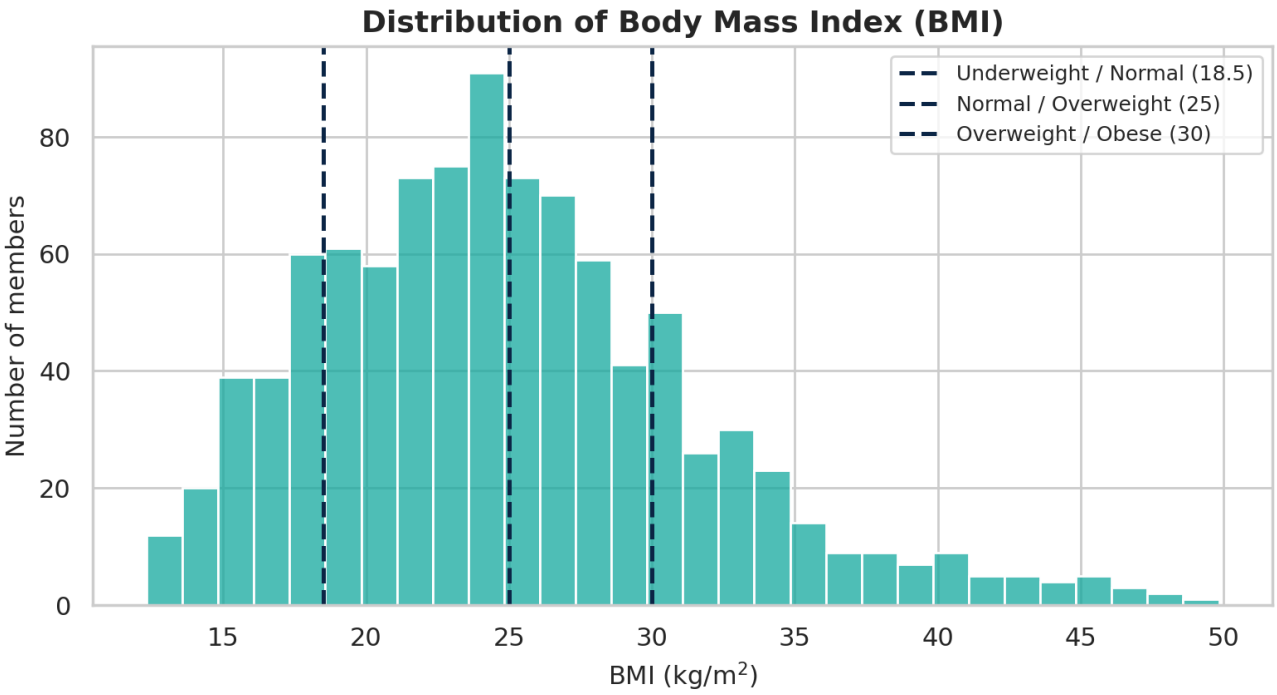


Figure 7. Distribution of Body Mass Index (BMI)

What it shows: BMI ranges from 12.3 to 49.8 with a mean of 24.91. The dashed lines mark WHO category boundaries; 44.7% of members fall in the overweight or obese range, and a long right tail reaches severe obesity.

OUTPUT | VISUAL

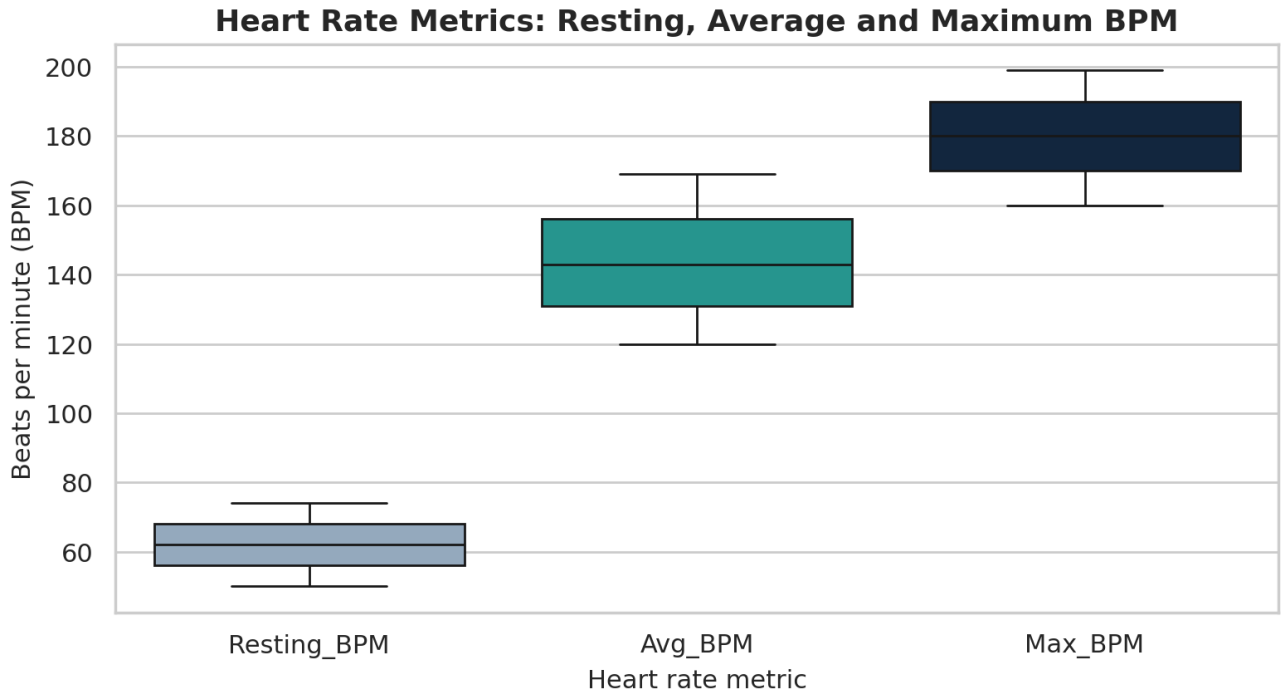


Figure 8. Heart-rate metrics: resting, average and maximum BPM

What it shows: Resting BPM averages 62.2 (range 50 to 74), average BPM during sessions 143.8 (120 to 169) and maximum BPM 179.9 (160 to 199). The average and maximum ranges overlap between 160 and 169 BPM, which is exactly where the 23 flagged records (average above maximum) occur.

10. Feature Engineering

Four new columns were added to a copy of the cleaned data. They convert raw measurements into categories and ratios that are easier to interpret and compare.

Feature	Definition	Code idea	Why it is useful
BMI_Category	WHO adult classes: Underweight (<18.5), Normal (18.5 to <25), Overweight (25 to <30), Obese (30 or more)	pd.cut on BMI with right=False	Turns a continuous number into groups for counting and comparing
Calories_Per_Hour	Calories_Burned / Session_Duration (hours)	Column division, rounded to 2 decimals	Measures workout intensity independent of session length
Workout_Duration_Category	Short (1 hour or less), Medium (over 1 to 1.5 hours), Long (over 1.5 hours)	pd.cut on duration with right=True	Compares short, medium and long sessions fairly
Water_Intake_Per_Kg	Water_Intake (liters) / Weight (kg)	Column division, rounded to 4 decimals	Adjusts hydration for body size

PYTHON CODE | STEP 6 (features)

```
df_fe = df_clean.copy()

# 6.1 BMI_Category (WHO adult classification)
df_fe["BMI_Category"] = pd.cut(df_fe["BMI"], bins=[0, 18.5, 25, 30, np.inf],
                               labels=["Underweight", "Normal", "Overweight", "Obese"],
                               right=False)

# 6.2 Calories_Per_Hour = calories burned / session duration (hours)
df_fe["Calories_Per_Hour"] = (df_fe["Calories_Burned"] / df_fe["Session_Duration (hours)"]).round(2)

# 6.3 Workout_Duration_Category (Short <= 1 h, Medium 1-1.5 h, Long > 1.5 h)
```

```
df_fe["Workout_Duration_Category"] = pd.cut(df_fe["Session_Duration (hours)"],
                                            bins=[0, 1, 1.5, np.inf],
                                            labels=["Short", "Medium", "Long"], right=True)

# 6.4 Water_Intake_Per_Kg = daily water intake (litres) / body weight (kg)
df_fe["Water_Intake_Per_Kg"] = (df_fe["Water_Intake (liters)"] / df_fe["Weight (kg)"]).round(4)

new_features = ["BMI_Category", "Calories_Per_Hour",
                "Workout_Duration_Category", "Water_Intake_Per_Kg"]
print("6.1 Sample of the engineered features (first 6 rows)")
print(df_fe[["BMI", "BMI_Category", "Calories_Burned", "Session_Duration (hours)",
             "Calories_Per_Hour"]].head(6).to_string())
print()
print(df_fe[["Session_Duration (hours)", "Workout_Duration_Category", "Water_Intake (liters)",
             "Weight (kg)", "Water_Intake_Per_Kg"]].head(6).to_string())

print("\n6.2 BMI_Category counts")
print(df_fe["BMI_Category"].value_counts().sort_index().to_string())
print("\n6.3 Workout_Duration_Category counts")
print(df_fe["Workout_Duration_Category"].value_counts().sort_index().to_string())
print("\n6.4 Summary of the numeric engineered features")
print(df_fe[["Calories_Per_Hour", "Water_Intake_Per_Kg"]].describe().T.round(4).to_string())
print("\nNew dataset shape:", df_fe.shape)
```

OUTPUT | TABULAR

```
6.1 Sample of the engineered features (first 6 rows)
     BMI BMI_Category  Calories_Burned  Session_Duration (hours)  Calories_Per_Hour
0  30.20        Obese             1313                      1.69             776.92
1  32.00        Obese              883                      1.30             679.23
2  24.71       Normal              677                      1.11             609.91
3  18.41  Underweight              532                      0.59             901.69
4  14.39  Underweight              556                      0.64             868.75
5  20.55       Normal             1116                      1.59             701.89

   Session_Duration (hours) Workout_Duration_Category  Water_Intake (liters)  Weight (kg)  Water_Intake_Per_Kg
0                      1.69                      Long                    3.5         88.3               0.0396
1                      1.30                    Medium                    2.1         74.9               0.0280
2                      1.11                    Medium                    2.3         68.1               0.0338
3                      0.59                     Short                    2.1         53.2               0.0395
4                      0.64                     Short                    2.8         46.1               0.0607
5                      1.59                      Long                    2.7         58.0               0.0466
```

OUTPUT | TABULAR

```
6.2 BMI_Category counts
BMI_Category
Underweight    168
Normal         370
Overweight     243
Obese          192
```

OUTPUT | TABULAR

```
6.3 Workout_Duration_Category counts
Workout_Duration_Category
Short     195
Medium    587
Long      191
```

OUTPUT | TABULAR

```
6.4 Summary of the numeric engineered features
                     count      mean      std       min       25%       50%       75%     max
Calories_Per_Hour    973.0  720.4214  86.9361  539.9000  655.5600  715.2700  781.9400  929.53
Water_Intake_Per_Kg  973.0    0.0374   0.0104    0.0156    0.0292    0.0374    0.0434    0.08
```

OUTPUT | TEXT

```
New dataset shape: (973, 19)
Saved chart: charts/09_bmi_category_counts.png
Saved chart: charts/10_calories_per_hour_by_workout.png
```

Interpretation

- **BMI_Category:** Underweight 168 (17.3%), Normal 370 (38.0%), Overweight 243 (25.0%), Obese 192 (19.7%).
- **Calories_Per_Hour:** mean 720.42 kcal/h (range 539.9 to 929.5). It is nearly the same for short, medium and long sessions (715.2, 722.5, 719.4), which explains why calories rise almost in proportion to duration.

- **Workout_Duration_Category:** 195 short, 587 medium and 191 long sessions; average calories rise from 549 to 907 to 1,265 kcal.
- **Water_Intake_Per_Kg:** mean 0.0374 litres per kg (about 37 ml per kg of body weight), with a range of 0.0156 to 0.0800.

OUTPUT | VISUAL

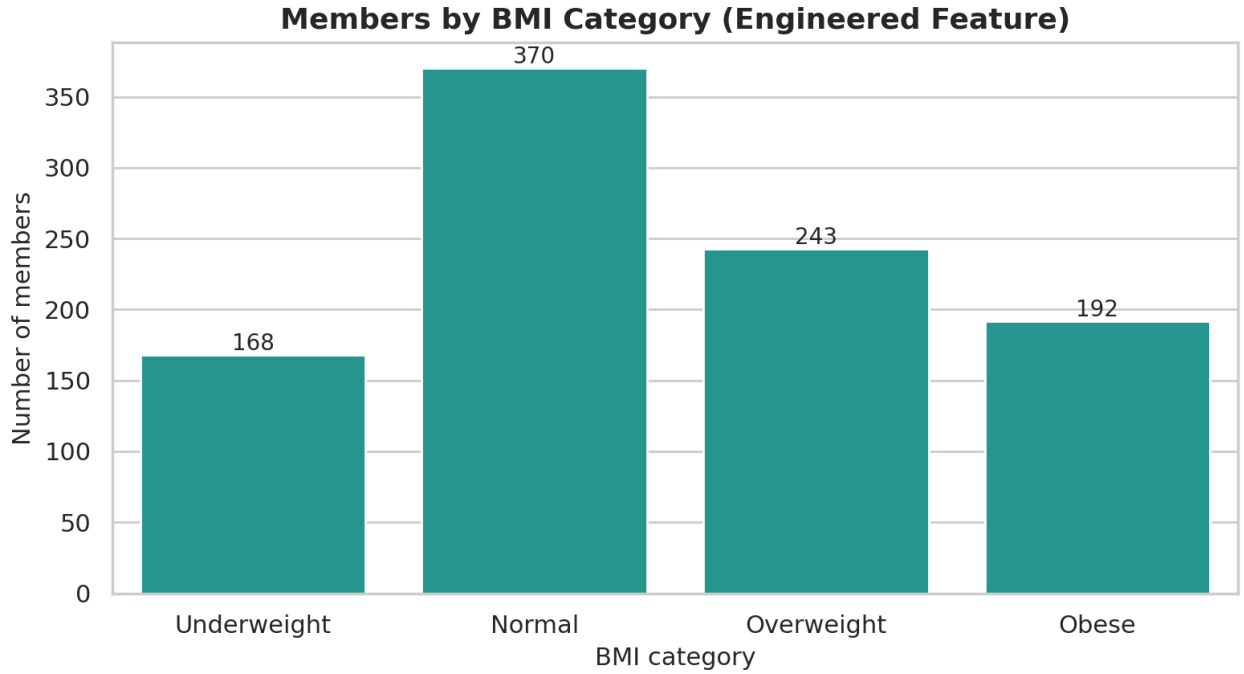


Figure 9. Members by BMI category (engineered feature)

What it shows: Normal weight is the largest group (370 members); together the overweight and obese groups (435 members) are larger than the normal group.

OUTPUT | VISUAL

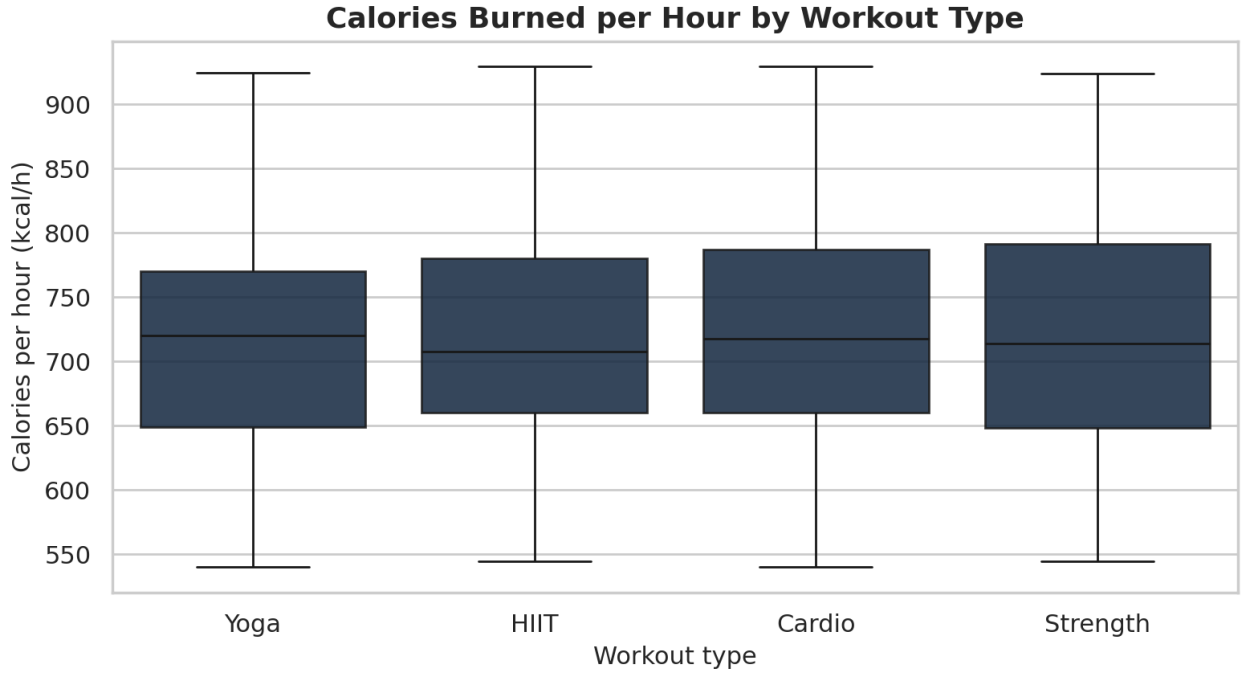


Figure 10. Calories burned per hour by workout type

What it shows: Intensity is similar across workouts (mean kcal/h: Strength 724, Cardio 724, Yoga 717, HIIT 717). All workout types show overlapping ranges, so no workout type is clearly more intense in this dataset.

11. Outlier Detection

The **Interquartile Range (IQR) method** (Tukey, 1977) flags a value as a potential outlier when it lies below **Q1 - 1.5 x IQR** or above **Q3 + 1.5 x IQR**, where Q1 and Q3 are the 25th and 75th percentiles and IQR = Q3 - Q1. It was applied to the 11 continuous original variables plus two engineered features.

PYTHON CODE | STEP 7

```
def iqr_bounds(series):
    q1, q3 = series.quantile(0.25), series.quantile(0.75)
    iqr = q3 - q1
    return q1, q3, iqr, q1 - 1.5 * iqr, q3 + 1.5 * iqr

outlier_cols = ["Age", "Weight (kg)", "Height (m)", "Max_BPM", "Avg_BPM", "Resting_BPM",
                "Session_Duration (hours)", "Calories_Burned", "Fat_Percentage",
                "Water_Intake (liters)", "BMI", "Calories_Per_Hour", "Water_Intake_Per_Kg"]
rows, flagged = [], pd.Series(False, index=df_fe.index)
for col in outlier_cols:
    q1, q3, iqr, lower, upper = iqr_bounds(df_fe[col])
    mask = (df_fe[col] < lower) | (df_fe[col] > upper)
    flagged |= mask
    rows.append({"Column": col, "Q1": q1, "Q3": q3, "IQR": iqr, "Lower": lower,
                 "Upper": upper, "Outliers": int(mask.sum()),
                 "Percent": round(mask.mean() * 100, 2)})
outlier_summary = pd.DataFrame(rows).set_index("Column")

print("7.1 IQR outlier summary (rule: below Q1 - 1.5*IQR or above Q3 + 1.5*IQR)")
print(outlier_summary.round(3).to_string())
print(f"\nRows with at least one flagged value: {int(flagged.sum())} of {len(df_fe)}")

# 7.2 Inspect the flagged BMI records instead of deleting them
_, _, _, bmi_low, bmi_high = iqr_bounds(df_fe["BMI"])
bmi_out = df_fe[(df_fe["BMI"] < bmi_low) | (df_fe["BMI"] > bmi_high)]
print("\n7.2 Flagged BMI records (highest first)")
print(bmi_out[["Age", "Gender", "Weight (kg)", "Height (m)", "BMI", "BMI_Category"]]
      .sort_values("BMI", ascending=False).head(10).to_string())

# 7.3 How much would removal change the result?
print("\n7.3 Effect of removing the BMI outliers")
print(f"Mean BMI with outliers    : {df_fe['BMI'].mean():.2f}")
bmi_kept = df_fe.loc[~df_fe.index.isin(bmi_out.index), "BMI"]
print(f"Mean BMI without outliers : {bmi_kept.mean():.2f}")
print("Decision: outliers are flagged and retained - they are plausible values, not errors.")
```

OUTPUT | TABULAR

```
7.1 IQR outlier summary (rule: below Q1 - 1.5*IQR or above Q3 + 1.5*IQR)
                             Q1        Q3      IQR    Lower     Upper  Outliers  Percent
Column
Age                      28.000    49.000   21.000   -3.500    80.500         0     0.00
Weight (kg)              58.100    86.000   27.900   16.250   127.850         9     0.92
Height (m)                1.620     1.800    0.180    1.350     2.070         0     0.00
Max_BPM                 170.000   190.000   20.000  140.000   220.000         0     0.00
Avg_BPM                 131.000   156.000   25.000   93.500   193.500         0     0.00
Resting_BPM              56.000    68.000   12.000   38.000    86.000         0     0.00
Session_Duration (hours)  1.040     1.460    0.420    0.410     2.090         0     0.00
Calories_Burned         720.000  1076.000  356.000  186.000  1610.000        10     1.03
Fat_Percentage           21.300    29.300    8.000    9.300    41.300         0     0.00
Water_Intake (liters)     2.200     3.100    0.900    0.850     4.450         0     0.00
BMI                      20.110    28.560    8.450    7.435    41.235        25     2.57
Calories_Per_Hour       655.560   781.940  126.380  465.990   971.510         0     0.00
Water_Intake_Per_Kg       0.029     0.043    0.014    0.008     0.065        13     1.34
```

OUTPUT | TEXT

```
Rows with at least one flagged value: 54 of 973
```

OUTPUT | TABULAR

7.2 Flagged BMI records (highest first)

	Age	Gender	Weight (kg)	Height (m)	BMI	BMI_Category
854	48	Male	129.2	1.61	49.84	Obese
945	35	Male	127.1	1.62	48.43	Obese
261	57	Male	126.8	1.63	47.72	Obese
710	38	Male	123.3	1.62	46.98	Obese
868	47	Male	127.8	1.65	46.94	Obese
470	34	Male	124.6	1.63	46.90	Obese
291	55	Male	128.4	1.68	45.49	Obese
133	21	Male	125.2	1.66	45.43	Obese
251	33	Male	117.0	1.61	45.14	Obese
687	18	Male	125.9	1.67	45.14	Obese

OUTPUT | TEXT

```
7.3 Effect of removing the BMI outliers
Mean BMI with outliers    : 24.91
Mean BMI without outliers : 24.39
Decision: outliers are flagged and retained - they are plausible values, not errors.
Saved chart: charts/11_outlier_boxplots.png
```

Actual outlier summary

Variables with at least one flagged value: **Weight (kg)** 9 (0.92%); **Calories_Burned** 10 (1.03%); **BMI** 25 (2.57%); **Water_Intake_Per_Kg** 13 (1.34%). The other 9 variables have none. In total **54 of 973 rows (5.5%)** contain at least one flagged value.

OUTPUT | VISUAL

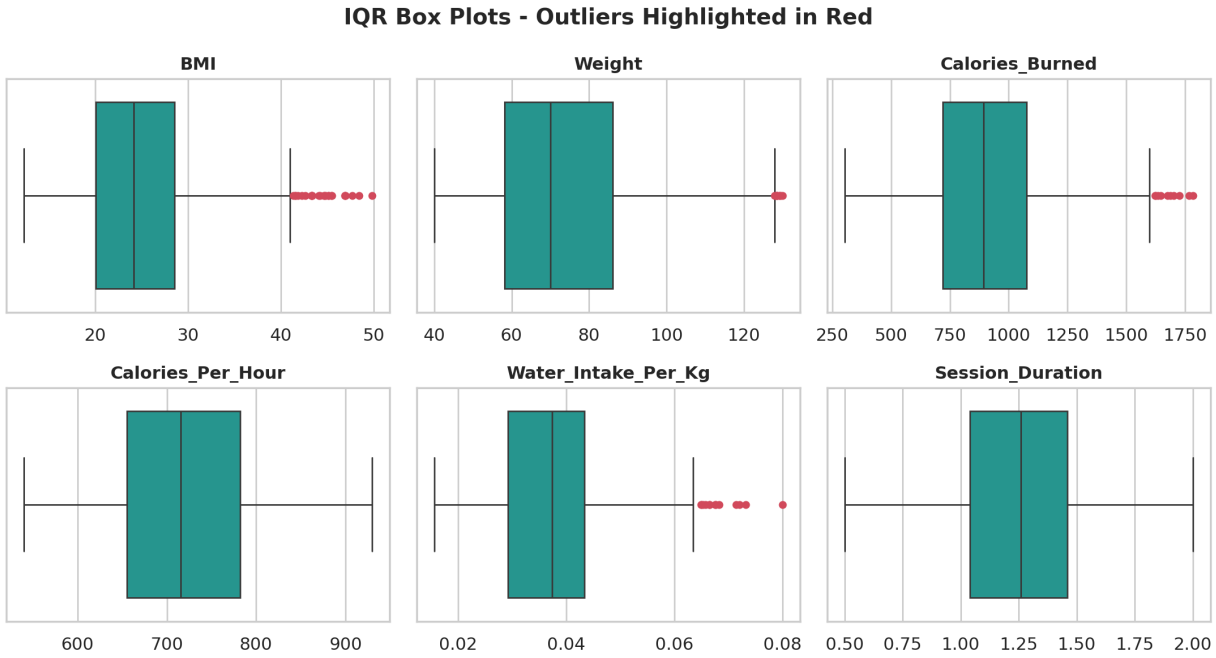


Figure 11. IQR box plots with outliers highlighted in red

What it shows: Box plots for six variables. Red points beyond the whiskers appear for BMI, Weight, Calories_Burned and Water_Intake_Per_Kg; Calories_Per_Hour and Session_Duration have none.

Why outliers should be investigated before being removed

- **An outlier is not an error.** The IQR rule only says a value is unusual relative to the middle of the data, not that it is wrong.
- **Here the outliers are plausible.** The 25 flagged BMI values are all in the obese range and come from members weighing 117 to 129 kg, which are real body sizes (Step 7.2).
- **Removal changes the picture.** Deleting the BMI outliers would lower the mean BMI from 24.91 to 24.39 and hide 25 real members, biasing conclusions about the group.

- **Investigation steps:** check the source, test whether weight and height make sense together, compare with related columns, and only then decide to keep, correct or remove.
- **Decision in this project:** all flagged rows were kept and documented; no value was altered.

12. Actual Findings and Results

All values below are calculated from the dataset by Steps 3 to 11. Step 11 prints the headline numbers directly from the data.

12.1 Headline results

Finding	Value
Dataset size	973 rows x 15 columns
Missing values / duplicate records	0 / 0
Heart-rate inconsistencies (Avg_BPM > Max_BPM)	23 records (2.4%), flagged and retained
Most common workout type	Strength (258 members, 26.5%)
Average calories burned	905.42 kcal (median 893, SD 272.64)
Average session duration	1.26 hours
Average BMI	24.91
Average heart rate (resting / average / maximum)	62.22 / 143.77 / 179.88 BPM
Correlation: duration vs calories	r = 0.908 (R-squared = 0.825); slope about 722 kcal per hour
Mean calories per hour	720.42 kcal/h
Mean water intake per kg	0.0374 litres/kg
Overweight or obese members	44.7%

12.2 Average calories burned by workout type

Workout type	Members	Avg calories (kcal)	Avg duration (h)	Avg Calories_Per_Hour
HIIT	221	925.81	1.29	716.51
Strength	258	910.70	1.26	724.00
Yoga	239	903.19	1.26	716.52
Cardio	255	884.51	1.22	723.85

Sorted from highest to lowest average calories. The whole spread is 41.3 kcal (4.6% of the mean), and ANOVA gives p = 0.42: the differences are consistent with chance.

12.3 Further findings

- **Gender:** men average 944.46 kcal versus 862.25 for women (9.5% higher), yet session durations are almost identical (1.25 h vs 1.26 h). The difference is statistically significant ($p < 0.001$). Men also have higher BMI (26.9 vs 22.7) and lower body-fat percentage (22.6% vs 27.7%).
- **Experience level:** average calories rise from 726 (level 1) to 902 (level 2) to 1,265 (level 3), and average duration from 1.01 h to 1.76 h. Level 3 members also have much lower body fat (14.8% vs 27.6% for level 1).
- **Overlapping drivers:** experience, workout frequency and session duration are strongly inter-correlated (for example Experience vs Frequency r = 0.84), so their separate effects on calories cannot be disentangled with simple correlations.
- **Other correlations with calories:** average heart rate r = 0.34, water intake 0.36, weight 0.10, BMI 0.06, age -0.15. Body size has almost no linear relationship with calories burned here.

- **BMI category:** average calories per hour is similar for Underweight (712), Normal (712) and Overweight (715) members and somewhat higher for Obese members (751 kcal/h).
- **Outliers:** 54 rows (5.5%) have at least one IQR-flagged value; BMI accounts for 25 of them. All were kept.

12.4 Answers to the research questions

RQ	Answer from the data
RQ1 Data quality	Clean in structure (0 missing, 0 duplicates, BMI consistent) but 23 records have Avg_BPM above Max_BPM.
RQ2 Member profile	Balanced workout mix (Strength most common at 26.5%), 52.5% male, mean BMI 24.91, average session heart rate 143.8 BPM.
RQ3 Calories	Mean 905.42 kcal. Workout type matters little (p = 0.42); gender, experience and duration matter a lot.
RQ4 Duration link	r = 0.908: strong positive linear relationship, about 722 kcal per additional hour.
RQ5 Engineered features	Calories_Per_Hour shows near-constant intensity across durations; BMI_Category and Duration_Category give interpretable groups; Water_Intake_Per_Kg normalises hydration by body size.
RQ6 Outliers	54 rows flagged by IQR; all are plausible values and were retained.

Comparison tables and charts that support these findings are reproduced in Section 14 (Code Outputs).

13. Source Code Appendix

The code below is the **complete and runnable** project script, printed exactly as it was executed to produce this report. It is also supplied as `FitPulse_360_Analysis.py`.

> **How to run it in Google Colab**
>
> 1) Open a new notebook at colab.research.google.com. 2) Upload `gym_members_exercise_tracking.csv` with the Files panel on the left. 3) Create one code cell per step below (each begins with a `# %%` marker; Step 1 includes the imports). 4) Run the cells in order. Charts are shown in the notebook and also saved in a `charts/` folder; the CSV exports appear in the Files panel.

Step 1: Setup, imports and data loading

Purpose: Imports the libraries, sets the plotting theme and colour palette, creates the charts folder, loads the CSV and prints shape and column names.

PYTHON CODE | STEP 1

```
# ============================================================================
# FitPulse 360 - Integrated Gym, Fitness and Activity Data Analytics
# Author  : Sarim Mohammed Maqsood Ahmed
# Runs in : Google Colab (or any Python 3.9+ environment)
# Dataset : gym_members_exercise_tracking.csv (Kaggle, valakhorasani)
# ============================================================================
import os
import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats   # used only for the significance checks in Step 4

warnings.filterwarnings("ignore")
sns.set_theme(style="whitegrid")
plt.rcParams.update({"savefig.dpi": 200, "axes.titleweight": "bold",
                     "axes.titlesize": 13, "axes.labelsize": 11})
NAVY, TEAL, SLATE = "#0B2545", "#13A89E", "#8DA9C4"
os.makedirs("charts", exist_ok=True)

# Google Colab: upload the CSV through the Files panel (left sidebar) or run:
#   from google.colab import files; files.upload()
```

```
CSV_PATH = "gym_members_exercise_tracking.csv"
df = pd.read_csv(CSV_PATH)

print(f"pandas {pd.__version__} | numpy {np.__version__} | seaborn {sns.__version__}")
print("Dataset loaded successfully.")
print(f"Rows: {df.shape[0]} | Columns: {df.shape[1]}")
print("Columns:")
for i, col in enumerate(df.columns, 1):
    print(f"{i:>3}. {col}")
```

Step 2: Data inspection

Purpose: Inspects the raw data: first rows (transposed so all 15 columns fit), data types and non-null counts, and unique values of categorical columns.

PYTHON CODE | STEP 2

```
# First rows are transposed so that all 15 columns fit on screen
print("First 5 rows (transposed):")
print(df.head().T.to_string())
print()
df.info()
print()
print("Unique values in categorical / ordinal columns:")
print("Gender           :", sorted(df["Gender"].unique().tolist()))
print("Workout_Type     :", sorted(df["Workout_Type"].unique().tolist()))
print("Experience_Level :", sorted(df["Experience_Level"].unique().tolist()))
```

Step 3: Data cleaning

Purpose: Cleans a copy of the data: missing values, duplicates, text standardisation, validity checks, BPM conflict inspection, BMI consistency check and export of the cleaned CSV.

PYTHON CODE | STEP 3

```
df_clean = df.copy()

# 3.1 Missing values (imputation runs only if a column actually has gaps)
missing = df_clean.isnull().sum()
print("3.1 Missing values per column")
print(missing.to_string())
print("Total missing values:", int(missing.sum()))
for col in df_clean.columns:
    if df_clean[col].isnull().any():
        if pd.api.types.is_numeric_dtype(df_clean[col]):
            df_clean[col] = df_clean[col].fillna(df_clean[col].median())
        else:
            df_clean[col] = df_clean[col].fillna(df_clean[col].mode()[0])
print("Missing values after treatment:", int(df_clean.isnull().sum().sum()))

# 3.2 Duplicate records
n_dup = int(df_clean.duplicated().sum())
print("\n3.2 Duplicate rows found:", n_dup)
df_clean = df_clean.drop_duplicates().reset_index(drop=True)
print("Rows after duplicate removal:", len(df_clean))

# 3.3 Text standardisation (remove accidental leading/trailing spaces)
text_cols = [c for c in df_clean.columns if not pd.api.types.is_numeric_dtype(df_clean[c])]
for col in text_cols:
    df_clean[col] = df_clean[col].astype(str).str.strip()
print("\n3.3 Text columns standardised:", text_cols)

# 3.4 Logical validity checks (0 means no problem found)
print("\n3.4 Logical validity checks")
checks = {
    "Non-positive numeric values": int((df_clean.select_dtypes("number") <= 0).sum().sum()),
    "Avg_BPM greater than Max_BPM": int((df_clean["Avg_BPM"] > df_clean["Max_BPM"]).sum()),
    "Resting_BPM greater than Avg_BPM": int((df_clean["Resting_BPM"] > df_clean["Avg_BPM"]).sum()),
    "Fat_Percentage outside 0-100": int((~df_clean["Fat_Percentage"].between(0, 100)).sum()),
}
for name, value in checks.items():
    print(f"{name:<36}: {value}")

# Inspect (but do not delete) records where average BPM exceeds maximum BPM
bpm_conflict = df_clean[df_clean["Avg_BPM"] > df_clean["Max_BPM"]]
print("\nRecords where Avg_BPM > Max_BPM (first 5 of", len(bpm_conflict), "):")
```

```
print(bpm_conflict[["Age", "Gender", "Max_BPM", "Avg_BPM", "Resting_BPM", "Workout_Type"]]
      .head(5).to_string())
print("Decision: flagged and retained - the source of the inconsistency cannot be verified.")

# 3.5 BMI consistency: recompute BMI = weight / height^2 and compare
bmi_check = (df_clean["Weight (kg)"] / df_clean["Height (m)"] ** 2).round(2)
bmi_gap = (bmi_check - df_clean["BMI"]).abs()
print(f"\n3.5 BMI recomputed from weight and height: max abs difference = {bmi_gap.max():.2f}")
print("Rows with a difference above 0.05:", int((bmi_gap > 0.05).sum()))

# 3.6 Export the cleaned dataset
df_clean.to_csv("fitpulse_cleaned_dataset.csv", index=False)
print("\nSaved: fitpulse_cleaned_dataset.csv", df_clean.shape)
```

Step 4: Exploratory data analysis - statistics

Purpose: Computes descriptive statistics, category counts, workout-type averages, correlations with Calories_Burned and three significance tests.

PYTHON CODE | STEP 4

```
num_cols = df_clean.select_dtypes(include="number").columns.tolist()

print("4.1 Descriptive statistics")
print(df_clean[num_cols].describe().T.round(2).to_string())

print("\n4.2 Category counts")
for col in ["Workout_Type", "Gender", "Experience_Level"]:
    counts = df_clean[col].value_counts()
    pct = (counts / len(df_clean) * 100).round(1)
    print(f"\n{col}")
    print(pd.DataFrame({"Count": counts, "Percent": pct}).to_string())

print("\n4.3 Average values by workout type")
by_workout = df_clean.groupby("Workout_Type")[
    ["Calories_Burned", "Session_Duration (hours)", "Avg_BPM", "BMI"]].mean().round(2)
print(by_workout.sort_values("Calories_Burned", ascending=False).to_string())

print("\n4.4 Correlation of every numeric column with Calories_Burned")
cal_corr = df_clean[num_cols].corr()["Calories_Burned"].drop("Calories_Burned")
print(cal_corr.sort_values(ascending=False).round(3).to_string())

def fmt_p(p):
    return "< 0.001" if p < 0.001 else f"= {p:.4f}"

print("\n4.5 Significance checks (alpha = 0.05)")
groups = [g["Calories_Burned"].values for _, g in df_clean.groupby("Workout_Type")]
f_stat, p_anova = stats.f_oneway(*groups)
print(f"One-way ANOVA, calories by workout type : F = {f_stat:.3f}, p {fmt_p(p_anova)}")
male = df_clean.loc[df_clean["Gender"] == "Male", "Calories_Burned"]
female = df_clean.loc[df_clean["Gender"] == "Female", "Calories_Burned"]
t_stat, p_gender = stats.ttest_ind(male, female, equal_var=False)
print(f"Welch t-test, calories male vs female   : t = {t_stat:.3f}, p {fmt_p(p_gender)}")
r_val, p_corr = stats.pearsonr(df_clean["Session_Duration (hours)"], df_clean["Calories_Burned"])
print(f"Pearson test, duration vs calories      : r = {r_val:.3f}, p {fmt_p(p_corr)}")
```

Step 5: Visualisations

Purpose: Creates eight charts (distribution, comparison, relationship, correlation, box plot and heart-rate views) and saves each as a high-resolution PNG.

PYTHON CODE | STEP 5

```
def save_chart(name):
    plt.tight_layout()
    plt.savefig(f"charts/{name}.png", bbox_inches="tight")
    plt.show()
    print(f"Saved chart: charts/{name}.png")

short = {c: c.split(" (")[0] for c in num_cols}   # shorter labels for plots

# Chart 1 - workout type distribution
order = df_clean["Workout_Type"].value_counts().index
plt.figure(figsize=(8, 4.5))
ax = sns.countplot(data=df_clean, x="Workout_Type", order=order, color=TEAL)
ax.bar_label(ax.containers[0], fontsize=10)
```

```
ax.set_title("Number of Members by Workout Type")
ax.set_xlabel("Workout type")
ax.set_ylabel("Number of members")
save_chart("01_workout_type_distribution")

# Chart 2 - calories distribution
plt.figure(figsize=(8, 4.5))
sns.histplot(df_clean["Calories_Burned"], bins=30, kde=True, color=NAVY)
plt.axvline(df_clean["Calories_Burned"].mean(), color=TEAL, lw=2.5,
            label=f"Mean = {df_clean['Calories_Burned'].mean():.0f}")
plt.axvline(df_clean["Calories_Burned"].median(), color="#D1495B", lw=2.5, ls="--",
            label=f"Median = {df_clean['Calories_Burned'].median():.0f}")
plt.title("Distribution of Calories Burned per Session")
plt.xlabel("Calories burned (kcal)")
plt.ylabel("Number of sessions")
plt.legend()
save_chart("02_calories_distribution")

# Chart 3 - average calories by workout type
avg_cal = df_clean.groupby("Workout_Type")["Calories_Burned"].mean().sort_values(ascending=False)
plt.figure(figsize=(8, 4.5))
ax = sns.barplot(x=avg_cal.index, y=avg_cal.values, color=NAVY)
ax.bar_label(ax.containers[0], fmt="%.1f", fontsize=10)
ax.set_title("Average Calories Burned by Workout Type")
ax.set_xlabel("Workout type")
ax.set_ylabel("Average calories burned (kcal)")
ax.set_ylim(0, avg_cal.max() * 1.15)
save_chart("03_avg_calories_by_workout")

# Chart 4 - session duration vs calories burned
r = df_clean["Session_Duration (hours)"].corr(df_clean["Calories_Burned"])
slope, intercept = np.polyfit(df_clean["Session_Duration (hours)"], df_clean["Calories_Burned"], 1)
dur = df_clean["Session_Duration (hours)"]
xs = np.linspace(dur.min(), dur.max(), 50)
plt.figure(figsize=(8, 4.8))
sns.scatterplot(data=df_clean, x="Session_Duration (hours)", y="Calories_Burned",
                hue="Gender", palette=[NAVY, TEAL], alpha=0.6, s=35)
plt.plot(xs, slope * xs + intercept, color="#D1495B", lw=2.5, label=f"Linear fit (r = {r:.2f})")
plt.title("Session Duration vs Calories Burned")
plt.xlabel("Session duration (hours)")
plt.ylabel("Calories burned (kcal)")
plt.legend()
save_chart("04_duration_vs_calories")

# Chart 5 - correlation heatmap
plt.figure(figsize=(10, 8))
cmap = sns.diverging_palette(250, 170, s=85, l=40, as_cmap=True)
sns.heatmap(df_clean[num_cols].corr().rename(index=short, columns=short), annot=True,
            fmt=".2f", cmap=cmap, center=0, vmin=-1, vmax=1, annot_kws={"size": 8},
            cbar_kws={"label": "Pearson correlation (r)"})
plt.title("Correlation Heatmap of Numeric Variables")
save_chart("05_correlation_heatmap")

# Chart 6 - calories by workout type and gender
plt.figure(figsize=(8, 4.8))
sns.boxplot(data=df_clean, x="Workout_Type", y="Calories_Burned", hue="Gender",
            palette=[NAVY, TEAL])
plt.title("Calories Burned by Workout Type and Gender")
plt.xlabel("Workout type")
plt.ylabel("Calories burned (kcal)")
plt.legend(title="Gender", loc="upper left", bbox_to_anchor=(1.01, 1))
save_chart("06_calories_by_workout_gender")

# Chart 7 - BMI distribution with WHO category boundaries
plt.figure(figsize=(8, 4.5))
sns.histplot(df_clean["BMI"], bins=30, color=TEAL)
for edge, label in [(18.5, "Underweight / Normal (18.5)"),
                    (25, "Normal / Overweight (25)"),
                    (30, "Overweight / Obese (30)")]:
    plt.axvline(edge, color=NAVY, ls="--", lw=1.8, label=label)
plt.title("Distribution of Body Mass Index (BMI)")
plt.xlabel("BMI (kg/m$^2$)")
plt.ylabel("Number of members")
plt.legend(fontsize=9)
save_chart("07_bmi_distribution")
```

```
# Chart 8 - heart rate metrics
hr = df_clean[["Resting_BPM", "Avg_BPM", "Max_BPM"]].melt(var_name="Metric", value_name="BPM")
plt.figure(figsize=(8, 4.5))
sns.boxplot(data=hr, x="Metric", y="BPM", palette=[SLATE, TEAL, NAVY])
plt.title("Heart Rate Metrics: Resting, Average and Maximum BPM")
plt.xlabel("Heart rate metric")
plt.ylabel("Beats per minute (BPM)")
save_chart("08_heart_rate_metrics")
```

Step 6: Feature engineering

Purpose: Engineers BMI_Category, Calories_Per_Hour, Workout_Duration_Category and Water_Intake_Per_Kg, summarises them and charts two of them.

PYTHON CODE | STEP 6

```
df_fe = df_clean.copy()

# 6.1 BMI_Category (WHO adult classification)
df_fe["BMI_Category"] = pd.cut(df_fe["BMI"], bins=[0, 18.5, 25, 30, np.inf],
                               labels=["Underweight", "Normal", "Overweight", "Obese"],
                               right=False)

# 6.2 Calories_Per_Hour = calories burned / session duration (hours)
df_fe["Calories_Per_Hour"] = (df_fe["Calories_Burned"] / df_fe["Session_Duration (hours)"]).round(2)

# 6.3 Workout_Duration_Category (Short <= 1 h, Medium 1-1.5 h, Long > 1.5 h)
df_fe["Workout_Duration_Category"] = pd.cut(df_fe["Session_Duration (hours)"],
                                            bins=[0, 1, 1.5, np.inf],
                                            labels=["Short", "Medium", "Long"], right=True)

# 6.4 Water_Intake_Per_Kg = daily water intake (litres) / body weight (kg)
df_fe["Water_Intake_Per_Kg"] = (df_fe["Water_Intake (liters)"] / df_fe["Weight (kg)"]).round(4)

new_features = ["BMI_Category", "Calories_Per_Hour",
                "Workout_Duration_Category", "Water_Intake_Per_Kg"]
print("6.1 Sample of the engineered features (first 6 rows)")
print(df_fe[["BMI", "BMI_Category", "Calories_Burned", "Session_Duration (hours)",
             "Calories_Per_Hour"]].head(6).to_string())
print()
print(df_fe[["Session_Duration (hours)", "Workout_Duration_Category", "Water_Intake (liters)",
             "Weight (kg)", "Water_Intake_Per_Kg"]].head(6).to_string())

print("\n6.2 BMI_Category counts")
print(df_fe["BMI_Category"].value_counts().sort_index().to_string())
print("\n6.3 Workout_Duration_Category counts")
print(df_fe["Workout_Duration_Category"].value_counts().sort_index().to_string())
print("\n6.4 Summary of the numeric engineered features")
print(df_fe[["Calories_Per_Hour", "Water_Intake_Per_Kg"]].describe().T.round(4).to_string())
print("\nNew dataset shape:", df_fe.shape)

# Chart 9 - BMI category counts
cat_counts = df_fe["BMI_Category"].value_counts().sort_index()
plt.figure(figsize=(8, 4.5))
ax = sns.barplot(x=cat_counts.index.astype(str), y=cat_counts.values, color=TEAL)
ax.bar_label(ax.containers[0], fontsize=10)
ax.set_title("Members by BMI Category (Engineered Feature)")
ax.set_xlabel("BMI category")
ax.set_ylabel("Number of members")
save_chart("09_bmi_category_counts")

# Chart 10 - calories per hour by workout type
plt.figure(figsize=(8, 4.5))
sns.boxplot(data=df_fe, x="Workout_Type", y="Calories_Per_Hour", color=NAVY,
            boxprops={"alpha": 0.85})
plt.title("Calories Burned per Hour by Workout Type")
plt.xlabel("Workout type")
plt.ylabel("Calories per hour (kcal/h)")
save_chart("10_calories_per_hour_by_workout")
```

Step 7: Outlier detection (IQR method)

Purpose: Applies the IQR rule to 13 variables, tabulates outliers, inspects flagged BMI records, measures the effect of removal and draws highlighted box plots.

PYTHON CODE | STEP 7

```
def iqr_bounds(series):
    q1, q3 = series.quantile(0.25), series.quantile(0.75)
    iqr = q3 - q1
    return q1, q3, iqr, q1 - 1.5 * iqr, q3 + 1.5 * iqr

outlier_cols = ["Age", "Weight (kg)", "Height (m)", "Max_BPM", "Avg_BPM", "Resting_BPM",
                "Session_Duration (hours)", "Calories_Burned", "Fat_Percentage",
                "Water_Intake (liters)", "BMI", "Calories_Per_Hour", "Water_Intake_Per_Kg"]
rows, flagged = [], pd.Series(False, index=df_fe.index)
for col in outlier_cols:
    q1, q3, iqr, lower, upper = iqr_bounds(df_fe[col])
    mask = (df_fe[col] < lower) | (df_fe[col] > upper)
    flagged |= mask
    rows.append({"Column": col, "Q1": q1, "Q3": q3, "IQR": iqr, "Lower": lower,
                 "Upper": upper, "Outliers": int(mask.sum()),
                 "Percent": round(mask.mean() * 100, 2)})
outlier_summary = pd.DataFrame(rows).set_index("Column")

print("7.1 IQR outlier summary (rule: below Q1 - 1.5*IQR or above Q3 + 1.5*IQR)")
print(outlier_summary.round(3).to_string())
print(f"\nRows with at least one flagged value: {int(flagged.sum())} of {len(df_fe)}")

# 7.2 Inspect the flagged BMI records instead of deleting them
_, _, _, bmi_low, bmi_high = iqr_bounds(df_fe["BMI"])
bmi_out = df_fe[(df_fe["BMI"] < bmi_low) | (df_fe["BMI"] > bmi_high)]
print("\n7.2 Flagged BMI records (highest first)")
print(bmi_out[["Age", "Gender", "Weight (kg)", "Height (m)", "BMI", "BMI_Category"]]
      .sort_values("BMI", ascending=False).head(10).to_string())

# 7.3 How much would removal change the result?
print("\n7.3 Effect of removing the BMI outliers")
print(f"Mean BMI with outliers    : {df_fe['BMI'].mean():.2f}")
bmi_kept = df_fe.loc[~df_fe.index.isin(bmi_out.index), "BMI"]
print(f"Mean BMI without outliers : {bmi_kept.mean():.2f}")
print("Decision: outliers are flagged and retained - they are plausible values, not errors.")

# Chart 11 - box plots with outliers highlighted
box_cols = ["BMI", "Weight (kg)", "Calories_Burned", "Calories_Per_Hour",
            "Water_Intake_Per_Kg", "Session_Duration (hours)"]
fig, axes = plt.subplots(2, 3, figsize=(11, 6))
for ax, col in zip(axes.ravel(), box_cols):
    sns.boxplot(x=df_fe[col], ax=ax, color=TEAL,
                flierprops={"markerfacecolor": "#D1495B", "markeredgecolor": "#D1495B",
                            "markersize": 4})
    ax.set_title(short.get(col, col), fontsize=11)
    ax.set_xlabel("")
fig.suptitle("IQR Box Plots - Outliers Highlighted in Red", fontsize=14, fontweight="bold")
save_chart("11_outlier_boxplots")
```

Step 8: Group comparisons

Purpose: Compares groups by gender, experience level, workout type, BMI category and duration category, and draws two comparison charts.

PYTHON CODE | STEP 8

```
print("8.1 Average values by gender")
print(df_fe.groupby("Gender")[["Calories_Burned", "Session_Duration (hours)", "Avg_BPM", "BMI",
                               "Fat_Percentage"]].mean().round(2).to_string())

print("\n8.2 Average values by experience level (1 = beginner, 3 = expert)")
print(df_fe.groupby("Experience_Level").agg(
    Members=("Age", "count"),
    Avg_Calories=("Calories_Burned", "mean"),
    Avg_Duration_h=("Session_Duration (hours)", "mean"),
    Avg_Fat_Pct=("Fat_Percentage", "mean")).round(2).to_string())

print("\n8.3 Average calories burned: workout type x gender")
print(df_fe.pivot_table(index="Workout_Type", columns="Gender", values="Calories_Burned",
                        aggfunc="mean").round(2).to_string())

print("\n8.4 By BMI category")
print(df_fe.groupby("BMI_Category", observed=True).agg(
    Members=("Age", "count"),
```

```
    Avg_Calories=("Calories_Burned", "mean"),
    Avg_Cal_Per_Hour=("Calories_Per_Hour", "mean")).round(2).to_string())

print("\n8.5 By workout duration category")
print(df_fe.groupby("Workout_Duration_Category", observed=True).agg(
    Members=("Age", "count"),
    Avg_Calories=("Calories_Burned", "mean"),
    Avg_Cal_Per_Hour=("Calories_Per_Hour", "mean")).round(2).to_string())

# Chart 12 - calories by experience level and by duration category
fig, axes = plt.subplots(1, 2, figsize=(11, 4.5))
exp_cal = df_fe.groupby("Experience_Level")["Calories_Burned"].mean()
sns.barplot(x=exp_cal.index, y=exp_cal.values, color=NAVY, ax=axes[0])
axes[0].bar_label(axes[0].containers[0], fmt="%.0f")
axes[0].set_title("Average Calories by Experience Level")
axes[0].set_xlabel("Experience level (1 = beginner, 3 = expert)")
axes[0].set_ylabel("Average calories burned (kcal)")
dur_cal = df_fe.groupby("Workout_Duration_Category", observed=True)["Calories_Burned"].mean()
sns.barplot(x=dur_cal.index.astype(str), y=dur_cal.values, color=TEAL, ax=axes[1])
axes[1].bar_label(axes[1].containers[0], fmt="%.0f")
axes[1].set_title("Average Calories by Duration Category")
axes[1].set_xlabel("Workout duration category")
axes[1].set_ylabel("Average calories burned (kcal)")
save_chart("12_experience_and_duration")

# Chart 13 - workout type x gender
plt.figure(figsize=(8, 4.5))
sns.barplot(data=df_fe, x="Workout_Type", y="Calories_Burned", hue="Gender",
            palette=[NAVY, TEAL], errorbar=None)
plt.title("Average Calories Burned: Workout Type and Gender")
plt.xlabel("Workout type")
plt.ylabel("Average calories burned (kcal)")
plt.legend(title="Gender", loc="upper left", bbox_to_anchor=(1.01, 1))
save_chart("13_workout_gender_comparison")
```

Step 9: Multi-chart dashboard

Purpose: Builds the multi-chart dashboard: a KPI strip plus six panels.

PYTHON CODE | STEP 9

```
fig = plt.figure(figsize=(10, 13.5))
gs = fig.add_gridspec(4, 2, height_ratios=[0.32, 1, 1, 1], hspace=0.42, wspace=0.55)

# KPI strip
kpi_ax = fig.add_subplot(gs[0, :])
kpi_ax.axis("off")
kpis = [("Records", f"{len(df_fe)}"),
        ("Avg calories", f"{df_fe['Calories_Burned'].mean():.0f} kcal"),
        ("Avg duration", f"{df_fe['Session_Duration (hours)'].mean():.2f} h"),
        ("Avg BMI", f"{df_fe['BMI'].mean():.1f}"),
        ("Avg BPM", f"{df_fe['Avg_BPM'].mean():.0f}")]
for i, (label, value) in enumerate(kpis):
    x = 0.01 + i * 0.2
    kpi_ax.add_patch(plt.Rectangle((x, 0.05), 0.18, 0.9, color=NAVY if i % 2 == 0 else TEAL,
                                   transform=kpi_ax.transAxes))
    kpi_ax.text(x + 0.09, 0.62, value, color="white", ha="center", fontsize=15,
                fontweight="bold", transform=kpi_ax.transAxes)
    kpi_ax.text(x + 0.09, 0.22, label, color="white", ha="center", fontsize=10.5,
                transform=kpi_ax.transAxes)

ax = fig.add_subplot(gs[1, 0])
sns.countplot(data=df_fe, x="Workout_Type", order=order, color=TEAL, ax=ax)
ax.bar_label(ax.containers[0], fontsize=9)
ax.set_title("A. Members by Workout Type", fontsize=11.5)
ax.set_xlabel("Workout type"); ax.set_ylabel("Members")

ax = fig.add_subplot(gs[1, 1])
sns.histplot(df_fe["Calories_Burned"], bins=25, kde=True, color=NAVY, ax=ax)
ax.axvline(df_fe["Calories_Burned"].mean(), color=TEAL, lw=2.2, label="Mean")
ax.set_title("B. Calories Burned Distribution", fontsize=11.5)
ax.set_xlabel("Calories burned (kcal)"); ax.set_ylabel("Sessions"); ax.legend(fontsize=9)

ax = fig.add_subplot(gs[2, 0])
sns.barplot(x=avg_cal.index, y=avg_cal.values, color=NAVY, ax=ax)
ax.bar_label(ax.containers[0], fmt="%.0f", fontsize=9)
```

```
ax.set_title("C. Average Calories by Workout", fontsize=11.5)
ax.set_xlabel("Workout type"); ax.set_ylabel("Avg calories (kcal)")
ax.set_ylim(0, avg_cal.max() * 1.15)

ax = fig.add_subplot(gs[2, 1])
sns.scatterplot(data=df_fe, x="Session_Duration (hours)", y="Calories_Burned", hue="Gender",
                palette=[NAVY, TEAL], alpha=0.55, s=22, ax=ax)
ax.plot(xs, slope * xs + intercept, color="#D1495B", lw=2.2, label=f"Fit (r = {r:.2f})")
ax.set_title("D. Duration vs Calories", fontsize=11.5)
ax.set_xlabel("Session duration (hours)"); ax.set_ylabel("Calories (kcal)")
ax.legend(fontsize=8.5)

ax = fig.add_subplot(gs[3, 0])
sns.barplot(x=cat_counts.index.astype(str), y=cat_counts.values, color=TEAL, ax=ax)
ax.bar_label(ax.containers[0], fontsize=9)
ax.set_title("E. Members by BMI Category", fontsize=11.5)
ax.set_xlabel("BMI category"); ax.set_ylabel("Members")
ax.tick_params(axis="x", labelsize=8)

ax = fig.add_subplot(gs[3, 1])
top = cal_corr.reindex(cal_corr.abs().sort_values(ascending=False).index).head(6).sort_values()
ax.barh([short[c] for c in top.index], top.values,
        color=[TEAL if v > 0 else "#D1495B" for v in top.values])
ax.set_title("F. Strongest Correlations with Calories", fontsize=11.5)
ax.set_xlabel("Pearson correlation (r)")
fig.suptitle("FitPulse 360 - Analytics Dashboard", fontsize=17, fontweight="bold",
             color=NAVY, y=0.94)
plt.savefig("charts/14_dashboard.png", bbox_inches="tight")
plt.show()
print("Saved chart: charts/14_dashboard.png")
```

Step 10: Export results

Purpose: Exports the feature-engineered dataset and outlier summary, and verifies the exports.

PYTHON CODE | STEP 10

```
df_fe.to_csv("fitpulse_feature_engineered_dataset.csv", index=False)
outlier_summary.round(4).to_csv("fitpulse_outlier_summary.csv")
print("Files exported:")
for f in ["fitpulse_cleaned_dataset.csv", "fitpulse_feature_engineered_dataset.csv",
          "fitpulse_outlier_summary.csv"]:
    print(f"  {f:<45} {os.path.getsize(f) / 1024:7.1f} KB")
print("\nConsistency check")
print("Cleaned rows == feature-engineered rows :", len(df_clean) == len(df_fe))
added = [c for c in df_fe.columns if c not in df_clean.columns]
print("Columns added by feature engineering    :", added)
reloaded = pd.read_csv("fitpulse_feature_engineered_dataset.csv")
print("Reloaded export shape                   :", reloaded.shape)
print("Charts saved                            :", len(os.listdir("charts")))
```

Step 11: Key findings summary

Purpose: Prints every headline number used in Section 12, calculated directly from the data.

PYTHON CODE | STEP 11

```
top_workout = df_fe["Workout_Type"].value_counts()
best_calories = avg_cal.index[0]
print("KEY FINDINGS (all values computed from the dataset)")
print("-" * 70)
print(f"Dataset size              : {df.shape[0]} rows x {df.shape[1]} columns")
print(f"Missing values            : {int(df.isnull().sum().sum())}")
print(f"Duplicate records         : {int(df.duplicated().sum())}")
print(f"Most common workout type  : {top_workout.index[0]} ({top_workout.iloc[0]} members)")
print(f"Average calories burned   : {df_fe['Calories_Burned'].mean():.2f} kcal")
print(f"Average session duration  : {df_fe['Session_Duration (hours)'].mean():.2f} hours")
print(f"Average BMI               : {df_fe['BMI'].mean():.2f}")
print(f"Average heart rate (avg)  : {df_fe['Avg_BPM'].mean():.2f} BPM")
print(f"Average max heart rate    : {df_fe['Max_BPM'].mean():.2f} BPM")
print(f"Average resting heart rate: {df_fe['Resting_BPM'].mean():.2f} BPM")
print(f"Highest-calorie workout   : {best_calories} ({avg_cal.iloc[0]:.2f} kcal on average)")
print(f"Lowest-calorie workout    : {avg_cal.index[-1]} ({avg_cal.iloc[-1]:.2f} kcal on average)")
print(f"Duration vs calories (r)  : {r:.3f}")
print(f"Calories vs Avg_BPM (r)   : {df_fe['Calories_Burned'].corr(df_fe['Avg_BPM']):.3f}")
print(f"Calories vs Weight (r)    : {df_fe['Calories_Burned'].corr(df_fe['Weight (kg)']):.3f}")
print(f"Mean calories per hour    : {df_fe['Calories_Per_Hour'].mean():.2f} kcal/h")
```

```
print(f"Mean water per kg         : {df_fe['Water_Intake_Per_Kg'].mean():.4f} litres/kg")
heavy = df_fe["BMI_Category"].isin(["Overweight", "Obese"])
print(f"Share overweight or obese : {heavy.mean() * 100:.1f}%")
```

14. Code Outputs

Every code step produces output of one or more of three kinds, labelled throughout this report: **text output** (printed messages), **tabular output** (pandas tables) and **visual output** (charts). The table shows where each step's output appears. Outputs not already shown in earlier sections are reproduced in full below.

Step	Description	Output types	Where shown
1	**Setup and load**	Text	14.1
2	**Data inspection**	Tabular, text	14.2
3	**Data cleaning**	Tabular, text	Section 7
4	**EDA statistics**	Tabular, text	Section 8
5	**Visualisations**	Visual (8 charts)	Section 9, Figures 1-8
6	**Feature engineering**	Tabular, text, visual (2)	Section 10, Figures 9-10
7	**Outlier detection**	Tabular, text, visual (1)	Section 11, Figure 11
8	**Group comparisons**	Tabular, visual (2)	14.3, Figures 12-13
9	**Dashboard**	Visual	Section 15, Figure 14
10	**Export**	Text	14.4
11	**Key findings**	Text	14.5

14.1 Step 1: Setup and load

OUTPUT | TEXT

```
pandas 3.0.2 | numpy 2.4.4 | seaborn 0.13.2
Dataset loaded successfully.
Rows: 973 | Columns: 15
Columns:
  1. Age
  2. Gender
  3. Weight (kg)
  4. Height (m)
  5. Max_BPM
  6. Avg_BPM
  7. Resting_BPM
  8. Session_Duration (hours)
  9. Calories_Burned
 10. Workout_Type
 11. Fat_Percentage
 12. Water_Intake (liters)
 13. Workout_Frequency (days/week)
 14. Experience_Level
 15. BMI
```

14.2 Step 2: Data inspection

OUTPUT | TABULAR

First 5 rows (transposed):

	0	1	2	3	4
Age	56	46	32	25	38
Gender	Male	Female	Female	Male	Male
Weight (kg)	88.3	74.9	68.1	53.2	46.1
Height (m)	1.71	1.53	1.66	1.7	1.79
Max_BPM	180	179	167	190	188
Avg_BPM	157	151	122	164	158
Resting_BPM	60	66	54	56	68
Session_Duration (hours)	1.69	1.3	1.11	0.59	0.64
Calories_Burned	1313	883	677	532	556
Workout_Type	Yoga	HIIT	Cardio	Strength	Strength
Fat_Percentage	12.6	33.9	33.4	28.8	29.2

```
Water_Intake (liters)           3.5     2.1     2.3       2.1       2.8
Workout_Frequency (days/week)     4       4       4         3         3
Experience_Level                  3       2       2         1         1
BMI                            30.2    32.0   24.71     18.41     14.39
```

OUTPUT | TEXT

```
<class 'pandas.DataFrame'>
RangeIndex: 973 entries, 0 to 972
Data columns (total 15 columns):
 #   Column                         Non-Null Count  Dtype
---  ------                         --------------  -----
 0   Age                            973 non-null    int64
 1   Gender                         973 non-null    str
 2   Weight (kg)                    973 non-null    float64
 3   Height (m)                     973 non-null    float64
 4   Max_BPM                        973 non-null    int64
 5   Avg_BPM                        973 non-null    int64
 6   Resting_BPM                    973 non-null    int64
 7   Session_Duration (hours)       973 non-null    float64
 8   Calories_Burned                973 non-null    int64
 9   Workout_Type                   973 non-null    str
 10  Fat_Percentage                 973 non-null    float64
 11  Water_Intake (liters)          973 non-null    float64
 12  Workout_Frequency (days/week)  973 non-null    int64
 13  Experience_Level               973 non-null    int64
 14  BMI                            973 non-null    float64
dtypes: float64(6), int64(7), str(2)
memory usage: 114.2 KB
```

OUTPUT | TEXT

```
Unique values in categorical / ordinal columns:
Gender           : ['Female', 'Male']
Workout_Type     : ['Cardio', 'HIIT', 'Strength', 'Yoga']
Experience_Level : [1, 2, 3]
```

14.3 Step 8: Group comparisons

OUTPUT | TABULAR

```
8.1 Average values by gender
        Calories_Burned  Session_Duration (hours)  Avg_BPM    BMI  Fat_Percentage
Gender
Female           862.25                     1.26   143.62  22.73           27.66
Male             944.46                     1.25   143.90  26.89           22.55
```

OUTPUT | TABULAR

```
8.2 Average values by experience level (1 = beginner, 3 = expert)
                 Members  Avg_Calories  Avg_Duration_h  Avg_Fat_Pct
Experience_Level
1                    376        726.38            1.01        27.63
2                    406        901.92            1.25        27.31
3                    191       1265.34            1.76        14.79
```

OUTPUT | TABULAR

```
8.3 Average calories burned: workout type x gender
Gender        Female    Male
Workout_Type
Cardio        834.79  933.08
HIIT          869.95  978.23
Strength      863.10  954.07
Yoga          886.12  916.79
```

OUTPUT | TABULAR

```
8.4 By BMI category
              Members  Avg_Calories  Avg_Cal_Per_Hour
BMI_Category
Underweight       168        827.38            711.64
Normal            370        927.98            711.99
Overweight        243        919.28            715.23
Obese             192        912.70            750.93
```

OUTPUT | TABULAR

```
8.5 By workout duration category
                          Members  Avg_Calories  Avg_Cal_Per_Hour
Workout_Duration_Category
Short                         195        548.55            715.24
Medium                        587        906.86            722.48
Long                          191       1265.34            719.40
Saved chart: charts/12_experience_and_duration.png
Saved chart: charts/13_workout_gender_comparison.png
```

Reading the tables: men burn more calories than women in every workout type (8.3); calories rise sharply with experience level (8.2) and with duration category (8.5), while calories per hour stays near 719 for all three duration categories.

OUTPUT | VISUAL

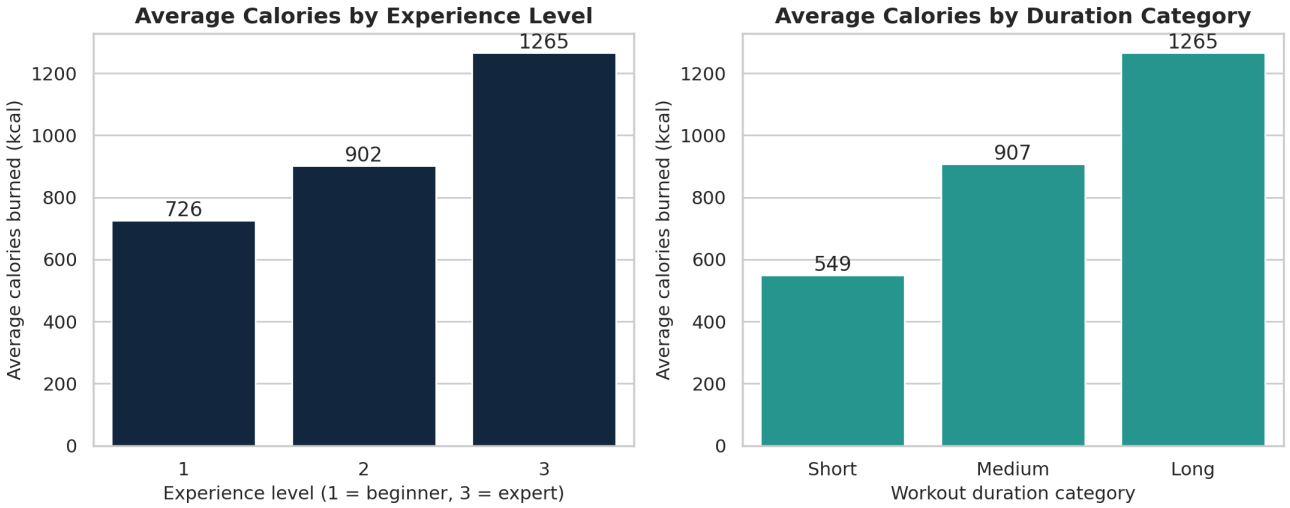


Figure 12. Average calories by experience level and by duration category

What it shows: Left: 726 kcal for level 1, 902 for level 2 and 1,265 for level 3. Right: 549, 907 and 1,265 kcal for short, medium and long sessions. Both show steep, steady increases.

OUTPUT | VISUAL

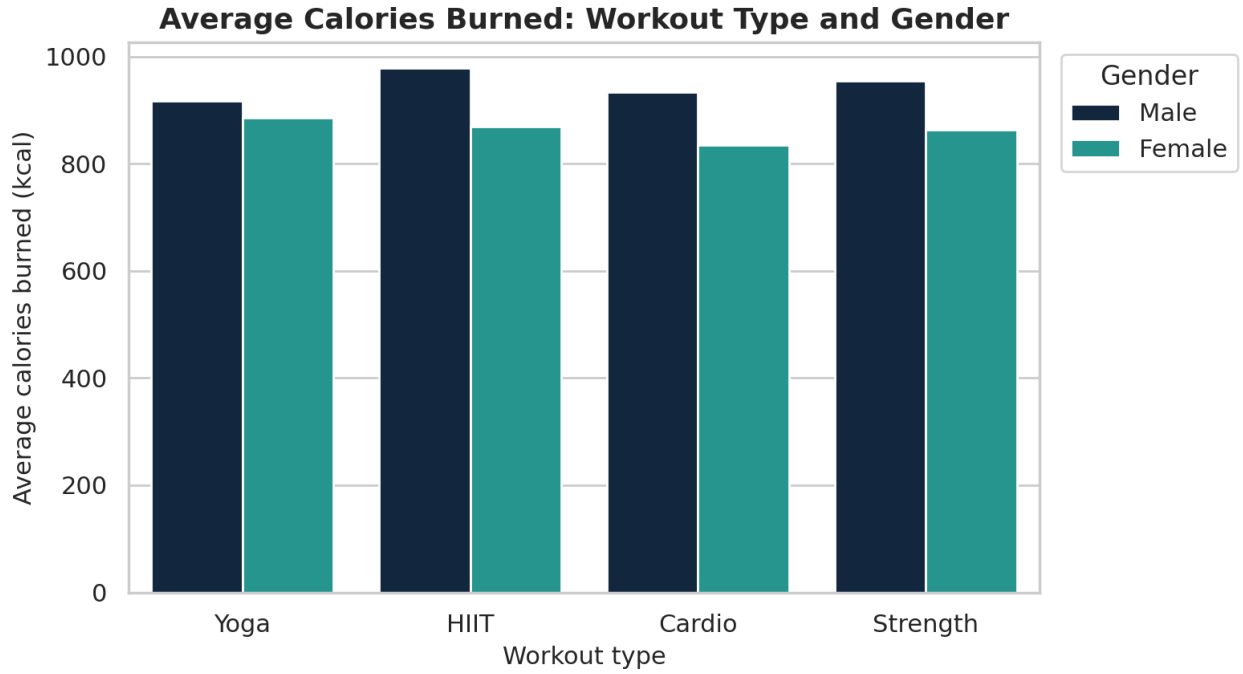


Figure 13. Average calories burned by workout type and gender

What it shows: Grouped bars confirm that male members average more calories than female members in every workout type (differences from 31 to 108 kcal).

14.4 Step 10: Export results

OUTPUT | TEXT

```
Files exported:
  fitpulse_cleaned_dataset.csv                    61.7 KB
  fitpulse_feature_engineered_dataset.csv         89.1 KB
  fitpulse_outlier_summary.csv                     0.7 KB

Consistency check
Cleaned rows == feature-engineered rows : True
Columns added by feature engineering    : ['BMI_Category', 'Calories_Per_Hour', 'Workout_Duration_Category', 'Water_Intake_Per_Kg']
Reloaded export shape                   : (973, 19)
Charts saved                            : 14
```

14.5 Step 11: Key findings summary

OUTPUT | TEXT

```
KEY FINDINGS (all values computed from the dataset)
----------------------------------------------------------------------
Dataset size              : 973 rows x 15 columns
Missing values            : 0
Duplicate records         : 0
Most common workout type  : Strength (258 members)
Average calories burned   : 905.42 kcal
Average session duration  : 1.26 hours
Average BMI               : 24.91
Average heart rate (avg)  : 143.77 BPM
Average max heart rate    : 179.88 BPM
Average resting heart rate: 62.22 BPM
Highest-calorie workout   : HIIT (925.81 kcal on average)
Lowest-calorie workout    : Cardio (884.51 kcal on average)
Duration vs calories (r)  : 0.908
Calories vs Avg_BPM (r)   : 0.340
Calories vs Weight (r)    : 0.095
Mean calories per hour    : 720.42 kcal/h
Mean water per kg         : 0.0374 litres/kg
Share overweight or obese : 44.7%
```

15. Dashboard

One image, generated by Step 9, combines a KPI strip (records, average calories, duration, BMI, heart rate) with six panels: A workout mix, B calories distribution, C average calories by workout, D duration vs calories, E BMI categories, F strongest correlations.

OUTPUT | VISUAL



Figure 14. FitPulse 360 analytics dashboard

What it shows: Balanced workout mix, calories centred near 905 kcal, small differences between workouts, a strong duration-calories link (r = 0.91), and duration as the dominant correlate of calories.

16. Project Limitations

- **Dataset size and scope:** 973 members in a single snapshot, with no dates or IDs. The analysis cannot show how any individual changes over time.
- **Unknown collection method:** the file does not say how members were selected or measured. Tight value ranges suggest the data may be curated or simulated, so results describe this dataset and may not describe real gym populations.
- **Missing information:** no diet, sleep, medical conditions, workout intensity settings, equipment, injury history or time of day. Activities such as dance or outdoor cardio are not represented.
- **Generalizability:** only four workout types, ages 18 to 59 and one source. Conclusions should not be applied to children, older adults, athletes or other populations.
- **Calorie estimates:** calories burned are probably estimates from a device or formula, not laboratory measurements. Calories per hour varies only between 540 and 930, which suggests a formula strongly tied to duration.
- **BMI interpretation:** BMI uses only weight and height and cannot tell muscle from fat. Muscular members can be classed as overweight. BMI_Category is a screening label, not a health diagnosis.
- **Correlation is not causation:** the strong link between duration and calories is partly mechanical (longer sessions burn more), and experience, frequency and duration move together, so their individual effects cannot be separated with correlations alone.
- **Data inconsistencies:** 23 records have Avg_BPM above Max_BPM; they were retained but may contain errors.
- **Analytical scope:** this is exploratory work. No predictive model, train/test split or multi-variable regression was built, and only three hypothesis tests were run.

17. Future Enhancements

Status

Everything in this section is **proposed future work**. None of it has been implemented in the current project.

Enhancement	Data needed	Possible approach
Gym data expansion	More members, repeated sessions, exercise-level logs (sets, reps, weight lifted)	Longitudinal analysis of progress; compare strength and cardio sessions
Dance and cardio tracking	Session logs for dance classes, running, cycling, swimming with intensity and duration	Add an Activity_Category column and compare calories per hour across activities
Real activity-tracker data	Exports from Fitbit, Garmin, Apple Health or Google Fit (heart rate, steps, sleep)	Clean time-series data, daily and weekly aggregation, resting heart-rate trends
Predictive modelling	The current dataset is enough to start	Linear regression and tree models to predict Calories_Burned with train/test split and cross-validation
Interactive dashboard	Cleaned and feature-engineered CSV files	Streamlit, Plotly Dash or Power BI with filters for gender, workout type and BMI category
Stronger statistics	Same data	Effect sizes, confidence intervals and multiple regression to separate the effects of duration, experience and frequency

18. Conclusion

FitPulse 360 delivers a complete exploratory data-analysis workflow on 973 gym-member records. The project loaded and inspected the data, verified its quality, cleaned and validated it, calculated descriptive and inferential statistics, produced 13 charts and a dashboard, engineered four new features, applied the IQR outlier method, compared groups and exported reproducible results.

The main findings are that session duration is the dominant driver of calories burned (r = 0.908), workout type makes little difference (a 4.6% spread, p = 0.42), and gender and experience level are associated with large differences in calories. The analysis also surfaced a data-quality problem (23 impossible heart-rate records) and handled it transparently rather than hiding it.

Skills demonstrated

- Python programming with Pandas, NumPy, Matplotlib and Seaborn in a Colab-compatible workflow
- Data cleaning, validation and quality reporting
- Exploratory analysis, correlation analysis and basic hypothesis testing
- Feature engineering with binning and ratio features
- Outlier detection and judgement about when not to delete data
- Data visualisation and dashboard design
- Honest reporting of limitations, and reproducible, documented code

As a beginner portfolio project, FitPulse 360 shows the full analytical process from raw CSV to documented insight, and it provides a solid base for the planned gym, dance, cardio and activity-tracker extensions.

19. References

[1] Kaggle. *Gym Members Exercise Dataset* (uploaded by valakhorasani). https://www.kaggle.com/datasets/valakhorasani/gym-members-exercise-dataset

[2] McKinney, W. (2010). Data Structures for Statistical Computing in Python. *Proceedings of the 9th Python in Science Conference*, 56-61. https://pandas.pydata.org

[3] Harris, C. R. et al. (2020). Array programming with NumPy. *Nature*, 585, 357-362. https://numpy.org

[4] Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90-95. https://matplotlib.org

[5] Waskom, M. L. (2021). seaborn: statistical data visualization. *Journal of Open Source Software*, 6(60), 3021. https://seaborn.pydata.org

[6] Virtanen, P. et al. (2020). SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, 17, 261-272. https://scipy.org

[7] Tukey, J. W. (1977). *Exploratory Data Analysis*. Addison-Wesley. (source of the 1.5 x IQR outlier rule)

[8] World Health Organization (2000). *Obesity: preventing and managing the global epidemic*. WHO Technical Report Series 894. (BMI classification used for BMI_Category)

[9] Google Colaboratory. https://colab.research.google.com

[10] ReportLab Open Source PDF Toolkit (used to generate this report). https://www.reportlab.com